

in28minutes.com · Interview Preparation Series

KUBERNETES

Interview Guide

10 Chapters · 109 Questions & Answers · June 2026

Table of Contents

1	Kubernetes Landscape	<i>6 Q&A</i>
2	Kubernetes Fundamentals	<i>6 Q&A</i>
3	Kubernetes Services and Networking	<i>9 Q&A</i>
4	More Kubernetes Fundamentals	<i>18 Q&A</i>
5	Kubernetes Storage	<i>11 Q&A</i>
6	Kubernetes Security	<i>10 Q&A</i>
7	Kubernetes Scheduling	<i>13 Q&A</i>
8	Kubernetes Observability	<i>9 Q&A</i>
9	More Kubernetes Essentials	<i>11 Q&A</i>
10	Kubernetes Interview - Diagrams	<i>16 Q&A</i>

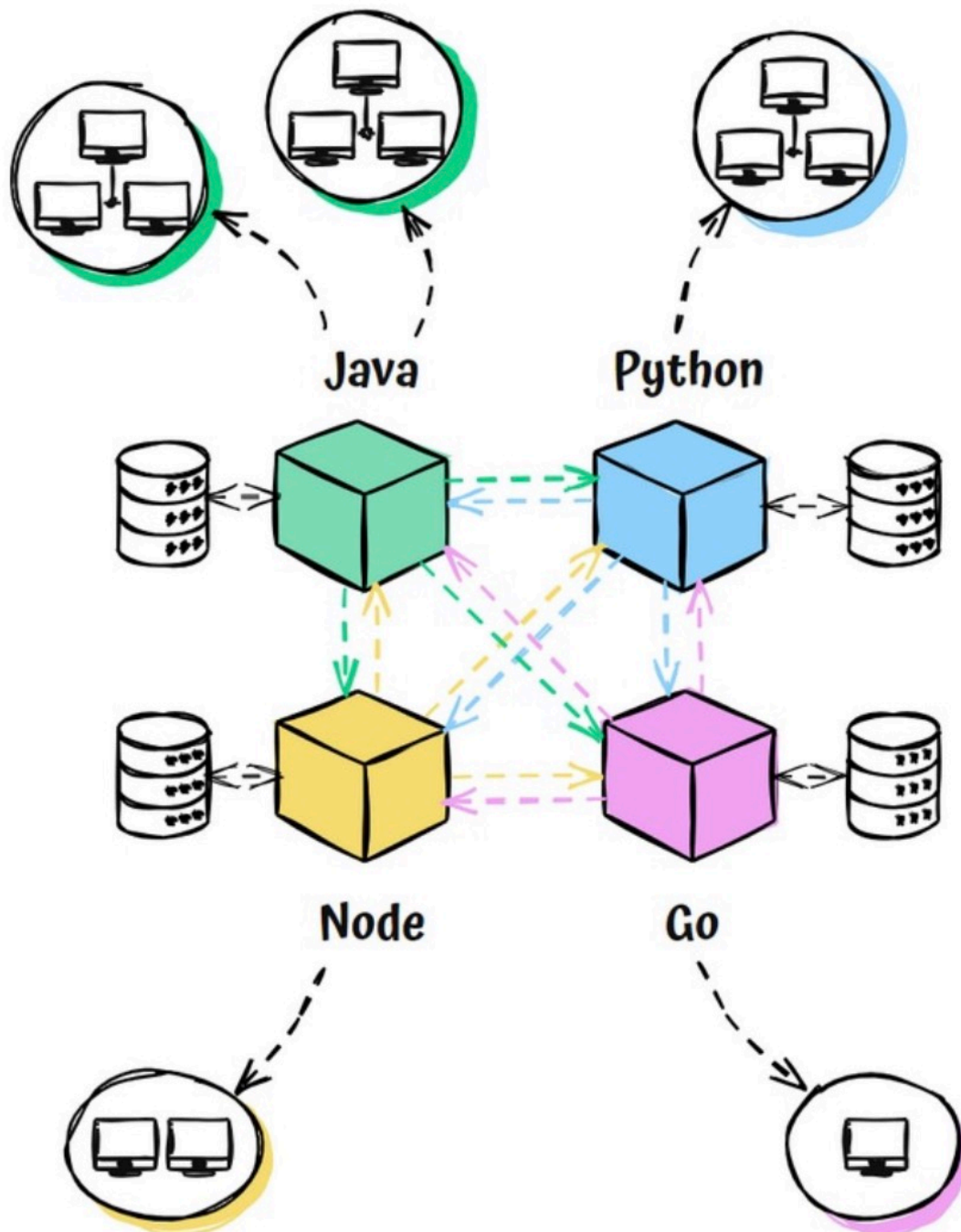
CHAPTER 1

Kubernetes Landscape

6 Questions & Answers

Q1

What are the Challenges faced in Microservices Architecture?

*Microservices*

- Scaling: Scaling individual components independently
- Service Discovery: Dynamically locating services in a distributed system
- Load Balancing: Distributing traffic among service instances
- Resiliency: Handling failures
- Release Management: Managing frequent releases
- Configuration Management: Maintaining different configurations for environments
- Security: Managing secrets, authentication, and authorization across services
- Monitoring and Logging: Collecting logs/traces across many services

Q2

Why is Kubernetes important for DevOps and Microservices?

What is Kubernetes?

- Definition: Kubernetes is an open-source container orchestration platform
- Goal: Automate deployment, scaling, and management of containerized applications
- Origin: Originally developed at Google, now maintained by CNCF (Cloud Native Computing Foundation)

How Does Kubernetes Work? (High-Level View)

- Create a Cluster - For example: Master Node + 4 Worker Nodes
- Deploy Microservice A - For example: 3 instances
Microservice A instances will be distributed across worker nodes
- Deploy Microservice B - For example: 4 instances
Microservice B instances will be distributed across worker nodes

Why is Kubernetes Important for DevOps?

- **Standardized Infrastructure:** Provides a uniform platform for deploying microservices regardless of language or framework
- **Standardized Automation of Deployment & Scaling:** Helps DevOps teams deploy and scale apps automatically
- **Environment Consistency:** Ensures consistent behavior across dev, test, and production
- **Supports Continuous Integration & Delivery (CI/CD):** Integrates well with Jenkins, GitHub Actions, etc. **Easy Rollouts & Rollbacks.**
- **Supports IaC:** Declarative configurations can be stored in Git and managed as code
- **Declarative State Management:** Define the desired state and let Kubernetes make it happen
- **YAML-Based Definitions:** Declare deployments, services, configs, etc. in version-controlled files

How Kubernetes Simplifies Microservices

- **Service Discovery & Load Balancing:** Built-in service abstraction (Built-in DNS and routing) makes service communication simple
- **Independent Scaling:** Scale individual microservices based on load
- **Observability Support:** Native integration with logging, monitoring, and tracing tools like Prometheus and Grafana

Kubernetes Enables High Availability and Resilience

- **Self-Healing:** Automatically restarts failed containers, reschedules on healthy nodes
- **Node Failover:** Reschedules workloads on surviving nodes during failures

Kubernetes Enables Efficient Resource Utilization

- **Isolated Deployments:** Each microservice runs in its own pod/container
- **Resource Limits:** Prevent noisy neighbor issues by defining CPU and memory limits
- **Auto-Scaling:** Adjust workload based on demand (HPA, VPA, Cluster Autoscaler)
- **Cost Optimization:** Better resource utilization reduces cloud spend
- **Namespaces:** Segment environments or teams on the same cluster

Kubernetes is Cloud & Platform Agnostic

- **Run Anywhere:** Supports on-premises, public cloud (AWS, Azure, GCP), hybrid, and edge
- **Avoids Vendor Lock-in:** Use the same Kubernetes setup across providers
- **Supports Multi-Cloud Strategy:** Run distributed clusters across different clouds

Q3**What Features does Kubernetes provide?**

- Automatic Scaling: Scales infrastructure and applications up or down based on demand
- Load Balancing: Distributes network traffic to maintain application stability
- Service Discovery: Automatically assigns DNS names to services & manages their IP addresses
- Self-Healing: Automatically restarts failed containers, reschedules them, & kills unresponsive ones
- Automated Rollouts & Rollbacks: Manages application updates with minimal downtime
- Storage Orchestration: Automatically mounts storage systems like local storage, public cloud providers, or network storage
- Secret & Configuration Management: Manages sensitive information & application configuration without rebuilding container images
- Multi-Tenancy: Isolates workloads in namespaces, allowing multiple teams to share a cluster
- Batch & Cron Workloads: Native Job & CronJob objects for oneoff or scheduled batch processing

Q4**How can you run Kubernetes?****Mode**

Minikube
Docker Desktop (with Kubernetes - macOS/Windows)
Kind (K8s-in-Docker)
Self-Managed
OpenShift
Cloud-managed
MicroK8s
K3s

Q5**Give an overview of Cloud-managed Kubernetes Options?**

- Cloud-specific services (NOT Kubernetes):

ECS: Amazon Elastic Container Service, a fully-managed container orchestrator with deep AWS integration

Fargate: Serverless (ECS or EKS) - abstracts away server and node management so you can focus on workloads or applications

- Cloud-native Kubernetes offerings:

EKS: Amazon Elastic Kubernetes Service, AWS-managed Kubernetes service

AKS: Azure Kubernetes Service, Microsoft-managed Kubernetes service

GKE: Google Kubernetes Engine, Google Cloud's Kubernetes service offering both Standard and Autopilot (serverless) modes

- Multi-cloud & hybrid platforms:

Amazon EKS Anywhere: Extension of EKS that lets you deploy and operate Kubernetes clusters on-prem or in other clouds

GKE Enterprise: Extend GKE to work in multiple environments, including on-prem, AWS, Azure, ..

Azure Arc-enabled Kubernetes: Brings Azure capabilities to Kubernetes clusters outside of Azure — including on-premises, AWS, Google Cloud, ..

Q6

How does Kubernetes Release Cycle work?

- Semantic Versioning: Kubernetes versions are expressed as x.y.z, where x is the major version, y is the minor version, and z is the patch version

Example: 1.33.2

- Release Cadence: Currently the project targets ~3 minor releases per year (every 4 months)

- Feature Phases: New features progress through alpha (testing), beta (more stable but evolving), and GA (General Availability - fully stable) stages

- Support Window: Each minor release currently receives patch & security updates for ~12 months

- Patch Releases: Issued as needed between major/minor releases (e.g., security or CVEs, critical bug fixes)

- Explore Release History: <https://kubernetes.io/releases/>

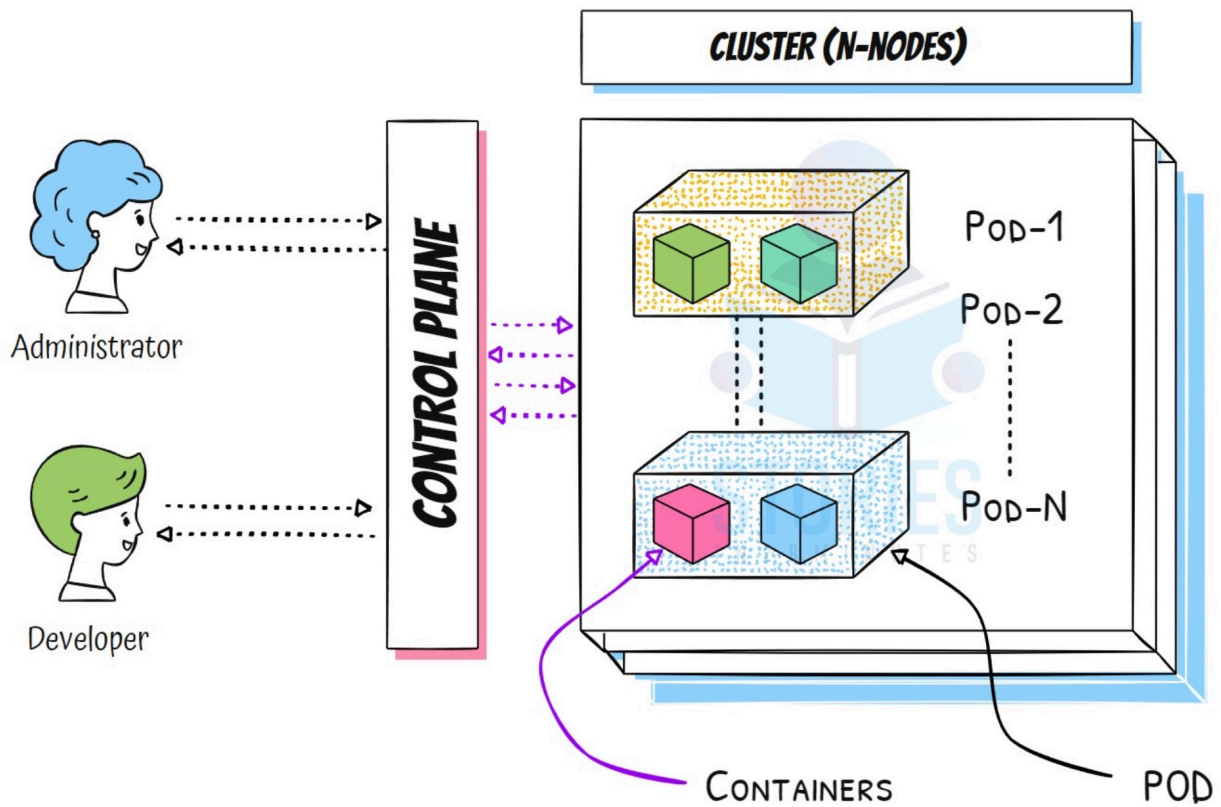
CHAPTER 2

Kubernetes Fundamentals

6 Questions & Answers

Q1

Explain Kubernetes Physical Architecture

*Kubernetes Physical View*

- **Types of nodes:** Architecture is divided into Master Nodes (Control Plane Nodes) & Worker Nodes (Data Plane Nodes)

Master Node: Responsible for managing the Kubernetes cluster & orchestrating the workloads

Worker Node: Machines where the actual application workloads (containers) run

Receives instructions from the Master Node

- **Common Components: (present on EVERY node - Master & Worker)**

Container Runtime: Responsible for running containers on all nodes

Kubelet: Ensures the containers in Pods are running and healthy as defined

Kube-Proxy: Facilitates load balancing and service discovery within the cluster

- **Master Nodes Components: (present only on Master Nodes)**

API Server: Central control point for the cluster, handling all REST API requests to manage Kubernetes objects

Scheduler: Allocates Pods to nodes based on current resource metrics and constraints

Controller Manager: Runs core controllers (e.g., Node Controller, Replication Controller) to maintain desired state.

Node Controller: Monitors the health of nodes

Replication Controller: Ensures the specified number of pod replicas are running at all times

Deployment Controller: Handles rolling updates, rollbacks

DaemonSet Controller, StatefulSet Controller, Job Controller, ...

Etc: A distributed key-value store that stores all cluster state data

Cloud Controller Manager: Interfaces with cloud provider APIs to handle cloud-specific resources like persistent disks and external load balancers

Q2

What is the role of Deployment, ReplicaSet and a Pod?

Let's consider deploying two version of a microservice v1 and v2:

Pod

- Smallest deployable unit in Kubernetes
- 1 or more containers: Runs one or more containers (typically one microservice version instance)

- Ephemeral: Pods can be created, destroyed, and replaced automatically

- Example:

v1 pod runs microservice:1.0 image

v2 pod runs microservice:2.0 image

ReplicaSet

- Ensures a specified number of pod replicas are running at any times
- Watches over pods and recreates them if they crash or are deleted
- Tied to a specific version of a microservice
- When you have two versions of a microservice, you will have two different ReplicaSet's - v1 ReplicaSet, v2 ReplicaSet

Deployment

- High-level abstraction that manages ReplicaSets
- Enables Rolling updates and Rollbacks
- Manages ReplicaSets and creates new ReplicaSets as needed (e.g., during updates)

Example: Single Deployment with Rolling Update

- Update the deployment from image v1 to v2
- Kubernetes:

Creates a v2 ReplicaSet

Gradually scales down the v1 ReplicaSet

v1 ReplicaSet(16), v2 ReplicaSet(4)

v1 ReplicaSet(12), v2 ReplicaSet(8)

v1 ReplicaSet(8), v2 ReplicaSet(12)

v1 ReplicaSet(4), v2 ReplicaSet(16)

v1 ReplicaSet(0), v2 ReplicaSet(16)

YAML / Kubernetes

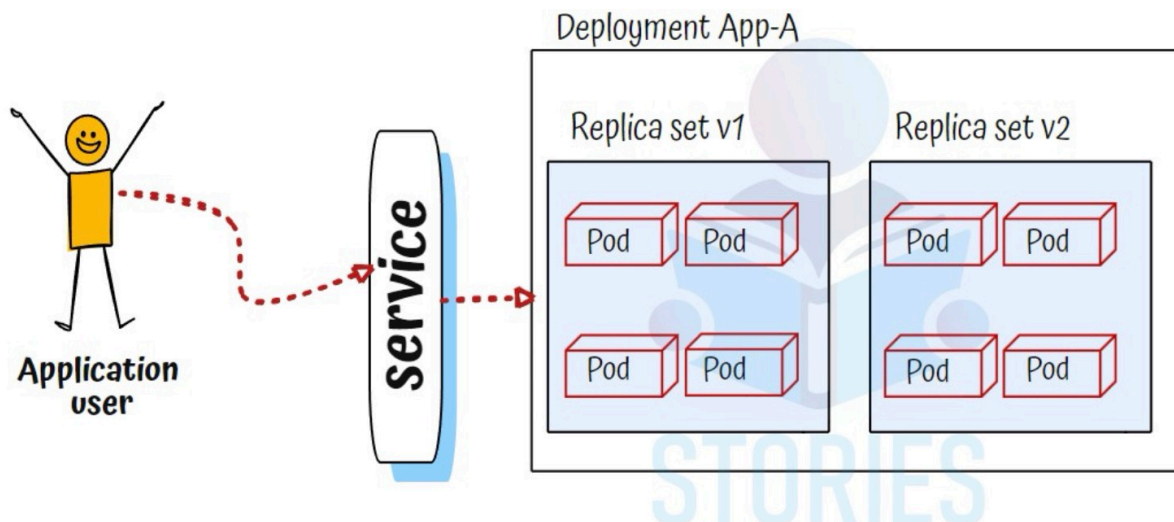
```
kubectl create deployment currency-exchange \
  --image=in28min/currency-exchange:0.0.1-RELEASE

# Step 2: Scale it to 16 replicas
kubectl scale deployment currency-exchange --replicas=16

# Step 3: (Optional) Update to a new version (e.g., newer tag)
kubectl set image deployment/currency-exchange \
  currency-exchange=in28min/currency-exchange:0.0.2-RELEASE
```

Code

```
Deployment
manages ReplicaSet (1 or more)
  maintains a set of Pods (single version)
  enables rolling updates between versions
```



Kubernetes Logical View

Kubernetes helps you manage multiple versions of a microservice using Deployments, ReplicaSets, and Pods, with a Service acting as the gateway between users and the running versions of a microservice.

Application User

- The end user interacts with the application through a stable endpoint.
- They do not know or care about internals of your Kubernetes architecture!

Service

- This acts as a stable IP and DNS name that routes traffic to the microservice versions
- In this case, the Service is:
 - Receiving requests from the user
 - Distributing them to Pods of the microservice

Deployment

- This is the high-level configuration that:
 - Creates and manages ReplicaSets
 - Manages the rollout and rollback process

ReplicaSet: v1 and v2

- Each ReplicaSet ensures a fixed number of Pods are running for a specific version.
- v1 represents older version (image: app:1.0)
- v2 newer version (image: app:2.0)

Pods

- Actual units of deployment that:
Run the application containers

Deployment creates ReplicaSets. ReplicaSets maintain Pods. The Service exposes Pods to users.

You can deploy multiple versions side-by-side and control which version users hit—all while maintaining zero downtime.

Q4

What happens in the background when you create a Deployment?

- **COMMAND EXECUTION:** The user runs the `kubectl create deployment` command with the name `currency-exchange` and a specified image
- **API SERVER INTERACTION:** The `kubectl` client sends a Deployment object to the Kubernetes API Server via a REST call
- **VALIDATION AND STORAGE:** The API Server authenticates the request, validates the Deployment spec, and stores it in `etcd`, the cluster's key-value store
- **DEPLOYMENT CONTROLLER TRIGGERED:** The Deployment Controller (part of the control plane) detects the new Deployment by watching for changes
- **REPLICASET CREATION:** The Deployment Controller creates a ReplicaSet that defines a Pod using the image `in28min/currency-exchange:0.0.1-RELEASE`
- **POD CREATION INITIATED:** The ReplicaSet requests the creation of 1 Pod (default) as specified in the template
- **SCHEDULER ASSIGNMENT:** The Scheduler assigns the Pod to an available Node in the cluster based on scheduling criteria
- **KUBELET AND CONTAINER START:** The kubelet on the selected Node receives the Pod spec, pulls the image, and starts the container via the container runtime
- **POD STATE: RUNNING:** Once the container is up, the Pod transitions to the Running state
- **DEPLOYMENT READY:** The Deployment is marked ready once the desired state (1 Pod running) is met

```
kubectl create deployment currency-exchange \
  --image=in28min/currency-exchange:0.0.1-RELEASE
```

Q5

What happens in the background when you Scale your Deployment?

- **COMMAND EXECUTION:** The user runs the `kubectl scale` command to update the `currency-exchange` Deployment and increase the replica count to 16
- **API SERVER INTERACTION:** `kubectl` sends a `PATCH` request to the Kubernetes API Server. The request updates the `.replicas` field of the Deployment object to 16
- **VALIDATION AND STORAGE:** The API Server authenticates the request, validates the update, and persists the modified Deployment object in `etcd`
- **DEPLOYMENT CONTROLLER TRIGGERED:** The Deployment Controller detects the change in the desired replica count and initiates reconciliation to meet the new target (16 Pods)
- **REPLICASET SCALING:** The existing ReplicaSet (associated with the Deployment) is scaled up from 1 to 16 replicas
- **POD CREATION INITIATED:** 15 additional Pods are requested to match the new desired state. Each Pod uses the same image and configuration as defined in the ReplicaSet.
- **SCHEDULER ASSIGNMENT:** The Scheduler assigns the new Pods to appropriate Nodes in the cluster, based on resource availability and other configuration
- **KUBELET AND CONTAINER START:** On each selected Node, the kubelet receives the Pod spec, pulls the container image (if not cached), and starts the container using the container runtime
- **PODS LIFECYCLE:** Each Pod transitions through its lifecycle phases: Pending ContainerCreating Running
- **DEPLOYMENT STATUS UPDATED:** Once all 16 Pods are running and pass readiness checks, the Deployment Controller updates the Deployment status to reflect the successful scaling.

YAML / Kubernetes

```
# Step 2: Scale it to 16 replicas
kubectl scale deployment currency-exchange --replicas=16
```

Q6

What happens in the background when you update the image in a Deployment?

- **COMMAND EXECUTION:** The user runs the `kubectl set image` command to update the image of the container named `currency-exchange` inside the Deployment `currency-exchange`, changing it to `in28min/currency-exchange:0.0.2-RELEASE`
 - **API SERVER INTERACTION:** `kubectl` sends a `PATCH` request to the Kubernetes API Server to update the Pod template in the Deployment spec with the new image
 - **VALIDATION AND STORAGE:** The API Server authenticates and validates the update, then stores the modified Deployment object in `etcd`
 - **DEPLOYMENT CONTROLLER TRIGGERED:** The Deployment Controller detects that the Pod template has changed (because the container image is different). This triggers a rolling update (default) process.
 - **NEW REPLICASET CREATED:** The Deployment Controller creates a new ReplicaSet corresponding to the new Pod template using the `0.0.2-RELEASE` image
 - **GRADUAL POD REPLACEMENT (Rolling Update):** The Deployment Controller begins replacing Pods from the old ReplicaSet (running `0.0.1-RELEASE`) with Pods from the new ReplicaSet:
 - A few new Pods are created with the new image
 - Once they are healthy and Ready, an equal number of old Pods are terminated
 - This continues until all old Pods are replaced
 - **SCHEDULER ASSIGNMENT:** As new Pods are created, the Scheduler assigns them to Nodes in the cluster based on availability
 - **KUBELET AND CONTAINER START:** For each new Pod, the kubelet on the assigned Node pulls the new image (if not cached) and starts the container
 - **ROLLING UPDATE STATUS MONITORING:** The Deployment Controller continuously monitors rollout status
 - **DEPLOYMENT STABILIZATION:** Once all Pods are updated to the new image and marked Ready, the Deployment is considered successfully rolled out
 - **OLD REPLICASET RETAINED (BY DEFAULT):** Kubernetes keeps the old ReplicaSet (scaled to zero) for rollback purposes, unless explicitly deleted.

YAML / Kubernetes

```
kubectl set image deployment/currency-exchange \
  currency-exchange=in28min/currency-exchange:0.0.2-RELEASE
```

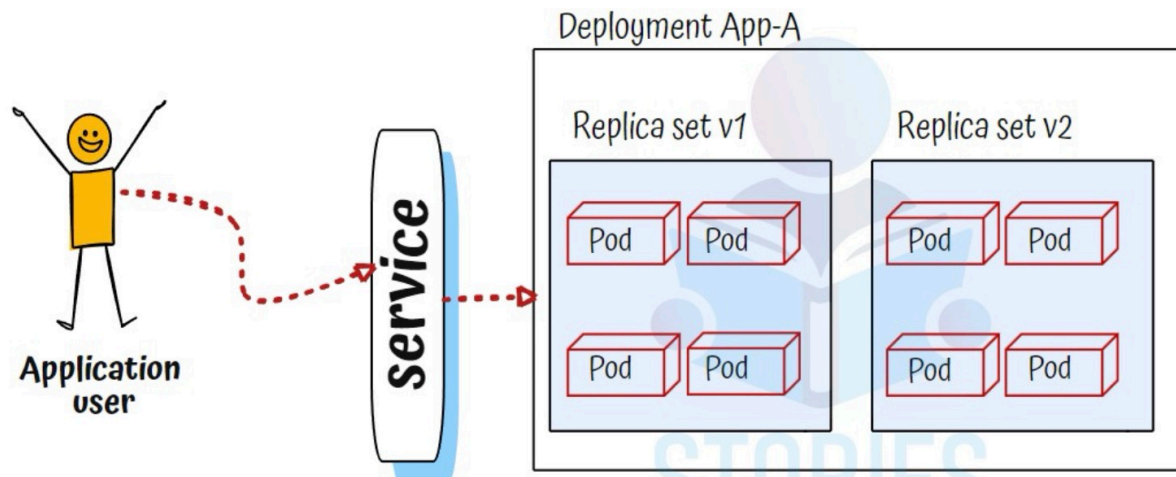

CHAPTER 3

Kubernetes Services and Networking

9 Questions & Answers

Q1

Why do we need a Kubernetes Service?

*Kubernetes Logical View*

- Stable Networking: Provides a stable IP and DNS name for a set of Pods, abstracting their ephemeral nature
- Load Balancing: Distributes network traffic evenly across multiple Pods to ensure reliability and performance

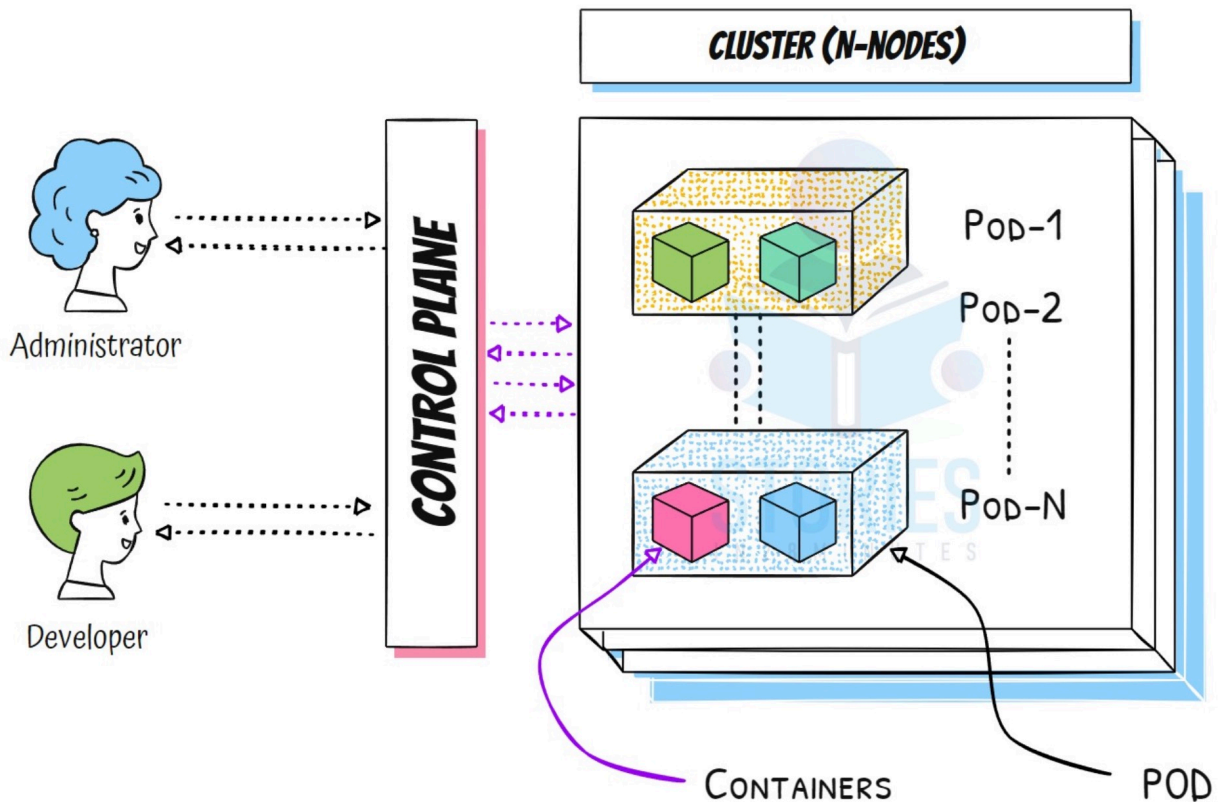
YAML / Kubernetes

```
kubectl expose deployment currency-exchange \
--type=LoadBalancer
```

Q2

What are the different Types of Services?

Kubernetes Physical View



- **ClusterIP**

Default Behavior: ClusterIP is the default service type in Kubernetes

Internal Access Only: Exposes the service only within the cluster

Used By Microservices: Ideal for internal communication between services

No External Access: Not reachable from outside the cluster

- **NodePort**

Port Binding: Exposes the service on a static port (30000–32767) on every node's IP

External Access via IP+Port: Accessible from outside via `http://<NodeIP>:<NodePort>`

Dev/Test Friendly: Common in non-production environments for quick access

Backed by ClusterIP: Uses a ClusterIP internally for routing to Pods

- **LoadBalancer**

Cloud Integration: Requests a cloud load balancer from supported platforms (AWS, GCP, Azure)

Public IP Assigned: Can make the service accessible via a public IP address

Production Ready: Common in production setups where stable internet access is needed

Uses NodePort + ClusterIP: Internally routes through NodePort to ClusterIP to reach Pods

Q3

Kubernetes Services Scenarios

- **Scenario: Why should you not use a Pod IP Address to communicate with a microservice?**

Pods are Ephemeral (not permanent): They can be terminated and replaced at any time by the scheduler (e.g., due to failure or scaling down)

New IP Address on Recreation: When a new Pod is created, it gets a new IP address

Breaks Communication: Any microservice hardcoded to communicate with a specific Pod IP will break when that Pod is replaced

- **Scenario: I'm creating a Service of LoadBalancer type. But I do NOT see a Load Balancer being created.**

Local Limitation: Kubernetes distributions running locally—like Minikube, Docker Desktop, or kind—lack integration with cloud providers, so they cannot automatically provision external LoadBalancers (tool specific workarounds exist!)

Cloud Dependency: In cloud environments, ensure that the Cloud Controller Manager is properly configured to provision and manage external LoadBalancers for LoadBalancer-type Services.

- **Scenario: I'm unable to access a NodePort service via `http://<NodeIP>:<NodePort>`**

No Public Access: NodeIP might not be publicly accessible (especially in private cloud or local clusters)

Firewall Blockage: Ports 30000–32767 might be blocked by firewalls or cloud security groups, preventing external access

When does NodePort work?

Same Network Access: If you're on the same network or subnet (e.g., your laptop and the cluster nodes are in the same corporate network or virtual private cloud), you can access the NodeIP directly

Public IP Consideration: If the Kubernetes node has a public IP, it can be accessed over the internet

Port Access Allowed: Access to NodePort is allowed by firewalls or cloud security groups

- **Scenario: I see another service type: ExternalName. What is it about? (This is different!)**

DNS Redirection: Used to map a Kubernetes service name to an external DNS (like `api.external.com`)

Internal Access to External Services: Allows in-cluster Pods to access external services using a Kubernetes-style service name

- **Scenario: Is a ClusterIP Created for NodePort or LoadBalancer?**

YES — Always Created: When you create a NodePort or LoadBalancer service, Kubernetes automatically creates a ClusterIP as the internal entry point for the service

Core Routing Layer: The ClusterIP is the core routing layer; NodePort and LoadBalancer are simply access mechanisms layered on top

- **Scenario: How Does Traffic Flow to different types of Services?**

Inside the Cluster: Pods can use the ClusterIP directly for communication

Outside the Cluster:

NodePort: Exposes the service via NodeIP:NodePort, which internally forwards traffic to the ClusterIP

LoadBalancer: Exposes the service via an external cloud load balancer, which routes traffic to the NodePort, then internally to the ClusterIP

Q4**What is the need for Fully Qualified Domain Name (FQDN)?**

- **Precise Location:** An FQDN specifies the precise location of a resource in the DNS hierarchy
- **No Ambiguity:** Avoids ambiguity by including all domain levels, from the specific host to the root
- **General FQDN Structure:** <hostname>.<subdomain>.<domain>.<tld>.
- **Example:** login.api.example.com.
- **Ends with a Dot (.)**
 - Root Indicator: Denotes the root of the DNS tree
 - Usually Omitted: Commonly omitted in day-to-day usage
- **DNS Resolution**
 - DNS Usage: An FQDN is used by DNS to resolve the resource to its corresponding IP address
 - Accurate Routing: Ensures accurate routing even across different namespaces, zones, or networks
- **Example: FQDN in Kubernetes**
 - Format: <service>.<namespace>.svc.cluster.local.
 - Example: currency-exchange.default.svc.cluster.local.
 - Cluster-Wide Access: Used to access services across the cluster
- **Scenario: When to Use FQDN in Kubernetes?**
 - Configs: Configuring service endpoints in config maps or external scripts

Q5**How does a Service help with Service Discovery?**

- **Stable Way to Access Dynamic Pods**

Ephemeral Pods: Pods are ephemeral — they can restart, move to other nodes, and receive new IPs

Stable Access Point: A Service provides a stable way to reach these changing Pods

- **Virtual IP (ClusterIP) and Load Balancing**

Virtual IP: A Service is assigned a ClusterIP — a virtual IP that does not change

Traffic Distribution: Requests sent to this IP are load-balanced across all healthy Pods selected by the Service

- **DNS-Based Service Discovery**

Automatic DNS Setup: Kubernetes automatically sets up DNS entries for each Service

Service FQDN Pattern: Kubernetes adds DNS records for every Service using the pattern:

`<service>.<namespace>.svc.cluster.local`

Name-Based Access: Any Pod can resolve and access other services using these DNS names — no need to hardcode IPs

- **Example: How Does a currency-conversion microservice talk to currency-exchange microservice?**

DNS Query Flow:

Pod (of currency-conversion microservice) issues DNS lookup (e.g., `currency-exchange.default.svc.cluster.local`)

CoreDNS resolves the service name and returns the ClusterIP address of the service

Service Name Format: Pods can resolve services using different formats like:

`currency-exchange` (within the same namespace)

`currency-exchange.default` (across namespaces)

`currency-exchange.default.svc.cluster.local` (fully qualified domain name)

Q6

Give a Practical Example of Service Discovery

Deployment with Two Instances

- **Deployment Applied:** A Deployment for currency-exchange is created with replicas: 2
- **Pod Creation:** A ReplicaSet with Two Pods (e.g., `currency-exchange-abcde`, `currency-exchange-fghij` with individual IPs `10.1.1.10`, `10.1.1.11`) is created

ClusterIP Service Created

- **ClusterIP Assigned:** A ClusterIP Service named currency-exchange is created. It selects the Pods using app=currency-exchange and gets a virtual IP, e.g., 10.96.0.10.
- **DNS Record Created:** CoreDNS creates a DNS mapping for currency-exchange.default.svc.cluster.local pointing to the ClusterIP
- **Endpoints object Created:** Kubernetes immediately creates an Endpoints object with the same name as the Service. The Endpoints Controller watches for Pods that match the Service's label selector. The matching Pod IPs and ports are populated into the Endpoints object.
- **iptables Rules Created:** kube-proxy on each node reads the Service and Endpoints, and sets up iptables rules to forward traffic from the ClusterIP to the IPs listed in the Endpoints object (i.e., to the 2 active Pods)

A New Instance is Created

- **New Pod Created:** The Deployment is scaled up, creating a new Pod (e.g., currency-exchange-xyz) with a new IP (e.g., 10.1.1.12)
- **Endpoint Object Updated:** Once the new Pod is Ready, it is automatically added to the Service's Endpoints object
- **iptables Refreshed:** kube-proxy detects the change and updates iptables rules to include the new Pod IP for load balancing

An Instance is Removed

- **Pod Terminated or Scaled Down:** One of the Pods (e.g., currency-exchange-fghij) is removed from the cluster
- **Endpoint Deregistration:** As soon as the Pod enters Terminating or fails readiness, it is removed from the Endpoints object
- **iptables Cleaned Up:** kube-proxy updates the iptables rules to exclude the removed Pod IP, ensuring no traffic is sent to it

Request Using FQDN

- **DNS Resolution by Client:** A client Pod issues a request to http://currency-exchange.default.svc.cluster.local — CoreDNS resolves it to the ClusterIP (e.g., 10.96.0.10)
- **Traffic Sent to ClusterIP:** The client sends a request to the ClusterIP
- **kube-proxy Load-Balancing:** Based on the iptables rules (which reflect the current Endpoints), traffic is forwarded to one of the Ready Pod IPs
- **Request Processed:** The selected Pod handles the request and sends a response back to the client

CoreDNS

- Purpose: Provides DNS-based service discovery within the Kubernetes cluster
- How It Works:
 - Watches the Kubernetes API Server for changes to Services
 - Resolves FQDNs like currency-exchange.default.svc.cluster.local to the Service's corresponding ClusterIP

Endpoints Object

- Purpose: Maintains a list of Pod IPs that are currently selected by the Service and are Ready to receive traffic
 - Dynamic Updates: Endpoints Controller (a control-plane component) watches for Services, Matching Pods and automatically updates the Endpoints Object
 - Used By: kube-proxy uses the Endpoints object to configure iptables (or IPVS) rules, enabling traffic routing to healthy Pod IPs behind a Service

kube-proxy

- Purpose: Acts as the traffic director for Services on each node
- How It Works:
 - Monitors the Kubernetes API for changes to Services and Endpoints
 - Dynamically updates iptables (or IPVS) rules to route Service traffic to healthy Pods
 - Ensures that traffic sent to a Service IP gets routed to one of the Ready Pods from the Endpoints object.

iptables

- Purpose: Store node-level routing rules to direct traffic from a ClusterIP (Service) to the correct Pod IPs
- How It Works: Rules are created and managed by kube-proxy

Q7

Why do we need an Ingress?

- Services: Multiple Microservices => Multiple Load Balancers: If each microservice is exposed externally using a Service of type LoadBalancer, a separate load balancer is provisioned per microservice (especially in cloud environments)
 - Ingress: Multiple Microservices => One Load Balancer: Routes incoming HTTP(S) traffic to appropriate backend services
 - Path-Based Routing: Forwards requests based on URL paths (e.g., /currency-exchange, /currency-conversion)
 - Host-Based Routing: Directs traffic based on hostnames (e.g., api.example.com, frontend.example.com)
- Complete Example: Ingress > Service > Deployment (Needs currency-exchange and currency-conversion services of Type ClusterIP or NodePort)

Code

```
- path: /currency-exchange
  pathType: Prefix
  backend:
    service:
      name: currency-exchange
      port:
        number: 80

- path: /currency-conversion
  pathType: Prefix
  backend:
    service:
      name: currency-conversion
      port:
        number: 80
```

Q8

Why do we need an Ingress Controller?

- **Ingress Is Only a Configuration**

No Execution Logic: An Ingress is a declarative object — it defines routing rules but doesn't perform any routing

Stored in etcd: Without a controller, the Ingress object simply sits in etcd and has no effect

- **Ingress Controller Applies the Rules**

Monitors Ingress Resources: Continuously watches for changes to Ingress objects in the cluster

Translates Rules: Converts Ingress definitions into actual configurations for tools like NGINX, HAProxy, or Envoy

Programs Networking: Sets up the underlying networking (e.g., iptables or proxy routes) to route traffic to Services

- **Without Ingress Controller**

No Traffic Routing: Ingress rules are not enforced, and external traffic cannot be routed

No Load Balancing: Advanced features like path-based routing won't work

- **Popular Ingress Controllers**

NGINX Ingress Controller (most widely used)

HAProxy

Traefik

Kong

AWS ALB Ingress Controller (for Amazon EKS)

Create Your Own: Custom controllers can be written to suit specific needs

Q9

Compare: Ingress vs LoadBalancer Service

Feature	Ingress
Purpose	Acts as an HTTP/HTTPS router for multiple Services
Scope	Can route to many Services based on host/path
Controller	Ingress Controller (e.g., NGINX, Traefik)
External IP	Shared across all routes via Ingress controller
Routing Rules	Path-based, host-based
Best For	Web applications, microservices behind shared endpoint

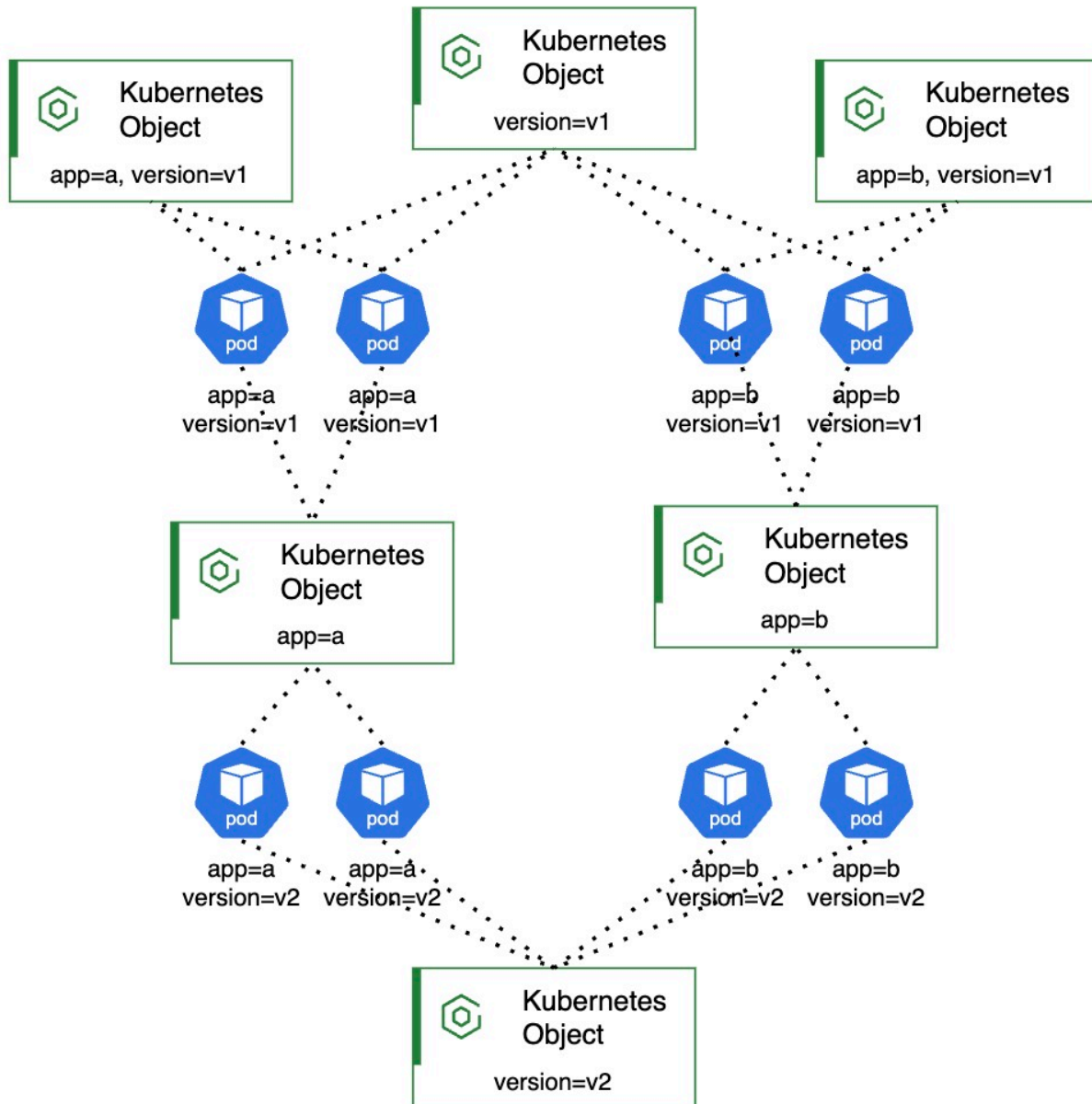
CHAPTER 4

More Kubernetes Fundamentals

18 Questions & Answers

Q1

Why do we need labels in Kubernetes?



Labels

- **Key Value Pairs:** Labels are key value pairs that can be attached with Kubernetes resources - Pods, Deployments, ReplicaSet, Service, ...

Assign multiple labels: You can assign multiple labels to a single resource

- **Why are Labels Needed?**

Logical Grouping of Resources: Labels allow you to group and organize resources (like Pods, Services, Deployments) - Grouping all resources belonging to currency-exchange microservice with app=currency-exchange

App Versioning: Use labels like version=v1 or version=v2 to track different versions of an application

Attach labels app=currency-exchange version=v1 with V1 and app=currency-exchange version=v2 with V2.

Route Traffic: Services use label selectors to route traffic to the correct set of Pods (A Service with selector: app=currency-exchange forwards traffic to both v1 and v2 pods. A Service with selector: app=currency-exchange & version=v1 forwards traffic only to v1 pods)

Organize Workloads: Labels can categorize workloads by team, app, or function (e.g., team=frontend, app=webapp)

- **Provides A Lot of Flexibility for Operations**

Example 1: Delete all staging resources using `kubectl delete pod -l environment=staging`

Example 2: Tools like Prometheus, Grafana can use labels to group, filter, and visualize metrics or logs - Show CPU usage grouped by app, team, or version

Example 3: Rollouts and Updates - Route 90% traffic to Pods with version=v1, and 10% to version=v2

Code

```
labels:  
  app: currency-exchange  
  environment: prod  
  tier: backend  
  version: v1  
  team: payments  
  billinggroup: finance
```

Q2

Explain an Example Deployment YAML

Simple Example

Another Example

[YAML](#) / [Kubernetes](#)

```

apiVersion: apps/v1                # Deployment API
kind: Deployment                    # Resource type
metadata:
  name: currency-exchange          # Deployment name
spec:
  replicas: 2                       # Desired pod count
  selector:                         # Selector to match pods
    matchLabels:
      app: currency-exchange
  template:
    metadata:
      labels:
        app: currency-exchange     # Pod label(s)
        version: v1
    spec:                           # POD SPEC
      containers:
        - name: currency-exchange   # Container name
          image:                    # Container image
            in28min/currency-exchange:0.0.1-RELEASE
          ports:
            - containerPort: 8000    # Exposed port

```

YAML / Kubernetes

```

apiVersion: apps/v1                # Deployment API
kind: Deployment                    # Resource type
metadata:
  name: currency-exchange          # Deployment name
spec:
  replicas: 2                       # Desired pod count
  selector:                         # Match these pods
    matchLabels:
      app: currency-exchange
  template:
    metadata:
      labels:
        app: currency-exchange     # Pod label
    spec:
      containers:
        - name: currency-exchange   # Container name
          image:                    # Container image
            in28min/currency-exchange:0.0.1-RELEASE
          imagePullPolicy: IfNotPresent # Use local image if exists
          ports:
            - containerPort: 8000    # Exposed container port
          env:                       # Environment variables
            - name: CURRENCY_HOST    # Sample env variable
              value: currency-db     # Set its value
          resources:                 # Resource requests/limits
            requests:
              memory: "128Mi"        # Minimum memory
              cpu: "250m"            # Minimum CPU
            limits:
              memory: "512Mi"        # Max memory allowed
              cpu: "500m"            # Max CPU allowed
          readinessProbe:            # Checks if pod is ready
            httpGet:
              path: /ready           # Health endpoint
              port: 8000             # Probe port
              initialDelaySeconds: 10 # Wait before first check
              periodSeconds: 10      # Check interval
          livenessProbe:             # Checks if pod is alive
            httpGet:
              path: /live            # Health endpoint
              port: 8000             # Probe port
              initialDelaySeconds: 15 # Wait before first check
              periodSeconds: 20      # Check interval

```

Q3**Explain an Example Service YAML**

- Frontend: How to receive Traffic?
- Backend: Which Pods to match against?

YAML / Kubernetes

```
apiVersion: v1                # Core API group
kind: Service                 # Resource type
metadata:
  name: currency-exchange     # Service name
spec:
  selector:
    app: currency-exchange    # Target pods
    version: v1
  ports:
    - port: 8000              # Service port
      targetPort: 8000        # Forward to this pod port
  type: LoadBalancer         # External access
```

Q4**What do selectors do in Kubernetes?****Example 1**

- Deployments and Services are mapped directly to Pods!

Example 2

- demo-v1 and demo-v2 are two separate Deployments with version labels
- demo-service sends traffic to both versions (v1 and v2)
- demo-service-v1 sends traffic only to Pods labeled version: v1

Selectors Give You a Lot of Flexibility

YAML / Kubernetes

```
apiVersion: apps/v1          # Deployment API
kind: Deployment              # Resource type
metadata:
  name: example-deployment    # Deployment name
spec:
  selector:
    matchLabels:              # Pod selector
      app: demo               # Match label app=demo
  template:
    metadata:
      labels:
        app: demo             # Pod label app=demo
---
apiVersion: v1                # Service API
kind: Service                  # Resource type
metadata:
  name: example-service        # Service name
spec:
  selector:
    app: demo                  # Match Pods with app=demo
```

[YAML / Kubernetes](#)


```

# Step 1: Deployment for version v1
apiVersion: apps/v1           # Deployment API
kind: Deployment              # Resource type
metadata:
  name: demo-v1               # Deployment name
spec:
  replicas: 2                  # Number of Pods
  selector:
    matchLabels:
      app: demo                # Select Pods with...
      version: v1              # app=demo
                                # version=v1
  template:
    metadata:
      labels:
        app: demo              # Labels for Pods
        version: v1            # App label
                                # Version label
---

# Step 2: Deployment for version v2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-v2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: demo
      version: v2
  template:
    metadata:
      labels:
        app: demo
        version: v2
---

# Step 3: Generic Service (targets both v1 and v2)
apiVersion: v1                # Service API
kind: Service                  # Resource type
metadata:
  name: demo-service          # Service name
spec:
  selector:
    app: demo                  # Match all versions
                                # with label app=demo
  ports:
    - port: 80                 # Service port
      targetPort: 8080          # Pod container port
  type: ClusterIP              # Internal-only access
---

# Step 4: Version-specific Service (targets only v1)
apiVersion: v1
kind: Service
metadata:
  name: demo-service-v1
spec:
  selector:
    app: demo
    version: v1                # Match v1 only
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP

```

Code

```

# Basic matchLabels
selector:
  matchLabels:
    app: demo                                # Match app=demo

---

# matchLabels with multiple key-values
# app=demo AND version=v1
# Best for Straightforward exact matches
selector:
  matchLabels:
    app: demo                                # Match app=demo
    version: v1                              # Match version=v1

---

# matchExpressions
# A flexible way to define label selectors
# Supports multiple operators:
# In, NotIn, Exists, DoesNotExist
# (app == blog OR app == shop) AND (version == v1)
selector:
  matchExpressions:
    - key: app
      operator: In
      values:
        - blog
        - shop                                # Match blog or shop
    - key: version
      operator: In
      values:
        - v1                                  # Match version=v1

---

# matchExpressions - NotIn operator
selector:
  matchExpressions:
    - key: env
      operator: NotIn
      values:
        - dev                                # Not environment=dev

---

# matchExpressions - Exists operator
selector:
  matchExpressions:
    - key: region
      operator: Exists                        # region key must exist

---

# matchExpressions - DoesNotExist operator
selector:
  matchExpressions:
    - key: debug
      operator: DoesNotExist                 # debug key must not exist

---

# matchExpressions - combine In and Exists
selector:
  matchExpressions:
    - key: app
      operator: In
      values:
        - demo
    - key: team
      operator: Exists                       # team label required

```

```

---

# matchExpressions - combine NotIn and Exists
selector:
  matchExpressions:
    - key: env
      operator: NotIn
      values:
        - staging
    - key: role
      operator: Exists          # role label required

---

# Combination: matchLabels + matchExpressions
# (app == api) AND (version in (v1, v2))
selector:
  matchLabels:
    app: api                    # matchLabels part
  matchExpressions:
    - key: version
      operator: In
      values:
        - v1                    # matchExpressions part
        - v2

```

Q5

What is the declarative approach in Kubernetes?

- Manifests as Source of Truth: Users create YAML or JSON files (called manifests) describing the desired state of Kubernetes resources like Deployments, Services, and ConfigMaps
 - Submitted via kubectl apply: The manifest files are submitted to the Kubernetes API Server using commands like `kubectl apply -f <file>.yaml`
 - API Server as Entry Point: The Kubernetes API Server validates and stores these resource definitions in etcd, the cluster's data store
 - Controller Loop Logic: Built-in controllers (like ReplicaSetController) continuously compare the desired state (from etcd) with the actual cluster state
 - Reconciliation in Action: When a mismatch is detected (e.g., a Pod crashes), the controller takes action (e.g., recreates the Pod) to bring the actual state back to the declared desired state
 - Declarative Updates: Changes to the manifest files and reapplying them cause Kubernetes to gradually reconcile the state
 - Stateless Execution: Controllers do not maintain execution history or steps — they always observe the current state and try to reconcile

Q6

Why is the declarative approach useful?

- **Desired State Management:** You declare what you want (e.g., 3 replicas), and Kubernetes works continuously to match the actual state with the desired state
- **Idempotent Configuration:** Reapplying the same YAML file has no side effects, ensuring repeatable and safe deployments
- **Version Control:** YAML files can be version-controlled, enabling audit trails and rollback
- **Environment Consistency:** The same configuration can be reused across dev, staging, and prod environments, ensuring predictable behavior
- **Simplified Troubleshooting:** Kubernetes exposes current and desired state through CLI, making it easier to diagnose problems
- **Seamless Automation:** Declarative configs integrate easily with CI/CD pipelines and automation tools

Q7

What are Namespaces?

- **Namespaces:** Kubernetes resources can be assigned a namespace

Default Namespace: If no namespace is specified, Kubernetes assigns objects to the default namespace

- **Logical Separation:** Provide a way to group cluster resources logically making management easier

- **Use cases:**

Multi-Tenancy: Enables isolation of resources between different teams or projects within the same cluster

Environment Segmentation: Isolates different environments (e.g., staging, testing) to prevent impact on production

Access Control: Allows fine-grained RBAC, restricting user access to specific namespaces

Example: Developers with full access in development but limited access in production

Resource Management: Sets resource limits for namespaces to prevent overuse

Example: CPU and memory quotas for different namespaces

Shared Resources: Common resources can be shared across multiple namespaces

Example: Monitoring, logging or ingress controllers

YAML / Kubernetes

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo
  namespace: my-namespace
# Other details

# View all namespaces
# kubectl get namespaces

# Run commands in a specific namespace
# kubectl get pods -n my-namespace

```

YAML / Kubernetes

```

# Namespace for Dev team workloads and resources
apiVersion: v1
kind: Namespace
metadata:
  name: dev-team
  labels:
    team: dev
    environment: development
---
# Namespace for Production workloads
apiVersion: v1
kind: Namespace
metadata:
  name: prod-team
  labels:
    team: prod
    environment: production
---
# ResourceQuota to cap total compute use in dev-team
# Helps prevent runaway resource consumption
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-limits
  namespace: dev-team
  labels:
    policy: enforced
spec:
  hard:
    requests.cpu: "2"           # Up to 2 CPUs can be requested
    requests.memory: 4Gi       # Up to 4Gi memory requested
    limits.cpu: "4"            # Absolute CPU limit for all pods
    limits.memory: 8Gi         # Absolute memory cap for all pods

```

Q8

Namespace Scenarios

- Scenario: What happens if I don't specify a namespace?
The object is created in the default namespace
- Scenario: Service Discovery fails across namespaces
Use FQDN: my-service.my-namespace.svc.cluster.local
- Scenario: A Pod/Service not found when searching
Possible Cause: You're looking in the wrong namespace
Fix: Add -n <namespace> or use --all-namespaces

Q9**What is the need for Gateway API?**

- **Modern Alternative to Ingress:** Gateway API is a new way to manage traffic into Kubernetes. It's more flexible and powerful than the old Ingress API.
- **Supports More Than HTTP:** It works with HTTP, HTTPS, TCP, UDP, gRPC, and TLS—not just web traffic.
- **Clear Team Roles:** Platform teams manage Gateways, while app teams define routes like HTTPRoute.
- **Reusable Gateway Configs:** GatewayClass lets you define shared settings for Gateways—just like StorageClasses for volumes.
- **Safe for Shared Clusters:** Routes from different namespaces can safely share the same Gateway.
- **Built for Extensibility:** It uses standard CRDs, so vendors like NGINX or Istio can add features without breaking compatibility.
- **Better Debugging Info:** It gives clear status updates, making it easier to monitor and troubleshoot.

Q10**Can you show a Gateway Multi-tenant Traffic Routing example?**

Let's create a shared Gateway in the infra namespace that can accept traffic from the internet and route it to multiple teams' services across namespaces.

[YAML / Kubernetes](#)

```

# OVERVIEW
# FLOW: Internet
# -> Gateway (port 80, in namespace 'infra')
# -> HTTPRoute (/app in namespace 'team-a',
#           /metrics in namespace 'team-b')
# -> Service (in team-a and team-b)
#
#
---
# WHAT: Defines a reusable gateway configuration
# BACKGROUND: Gateway controller detects it
# The controller takes ownership of any Gateway objects
# that reference this GatewayClass.
# No infra is provisioned YET!
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: shared-gateway-class
spec:
  controllerName: nginx.org/gateway-controller

---
# WHAT: Shared external entry point defined in 'infra'
# BACKGROUND: Gateway Controller configures the
# Gateway (NGINX, Envoy, Cloud LoadBalancer, ..)
# to listen on the configured port
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: shared-gateway
  namespace: infra
spec:
  gatewayClassName: shared-gateway-class

  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
          from: All
      # Allows HTTPRoute objects from any namespace to bind

---
# WHAT: Routing rules created in 'team-a' namespace
# BACKGROUND: This HTTPRoute links the shared Gateway
# Gateway (NGINX, Envoy, Cloud LoadBalancer, ..)
# is updated
# Match any HTTP request with prefix /app
# Forward requests to `my-app-service`
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: route-to-app
  namespace: team-a
spec:
  parentRefs:
    - name: shared-gateway
      namespace: infra
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /app
      backendRefs:
        - name: my-app-service # PRE EXISTING SERVICE
          port: 80

---
# WHAT: Second route from 'team-b' namespace
# BACKGROUND: This HTTPRoute also attaches to the
# shared Gateway and matches /metrics. Shows how
# multiple teams can use one Gateway.
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute

```

```

metadata:
  name: route-to-metrics
  namespace: team-b
spec:
  parentRefs:
    - name: shared-gateway
      namespace: infra
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /metrics
      backendRefs:
        - name: metrics-service # PRE EXISTING SERVICE
          port: 80

```

Q11

How is Gateway different from Ingress?

Feature	Ingress
Protocol Support	HTTP/HTTPS only
Path Matching	Yes
Role Separation	No
Extensibility (CRDs)	No
Multi-Tenancy Support	Limited
Status Reporting	Minimal

Q12

What are the scaling options in Kubernetes?

Why Scaling in Kubernetes?

- Ensure availability and performance as load varies
- Optimize resource usage and cost
- Automate infrastructure management
- Prevent over/under-provisioning

Scaling Options

Scaling Layer	What It Scales
Pod-level (Horizontal)	Number of Pod replicas
Pod-level (Vertical)	CPU/Memory of each Pod
Cluster-level	Number of Nodes in cluster

Q13

What is Horizontal Pod AutoScaler (HPA)?

- Purpose: Automatically scales the number of Pod replicas in a Deployment based on observed metrics (CPU, memory, or custom)
- Requirement: Metrics Server add-on should be installed & running to expose Pod or Node CPU/memory usage

Example:

YAML / Kubernetes

```
# HPA for scaling the 'web-app' deployment based on CPU
apiVersion: autoscaling/v2
kind: HorizontalPodAutoScaler
metadata:
  name: web-app-hpa          # HPA resource name
spec:
  scaleTargetRef:           # Target resource to scale
    apiVersion: apps/v1     # API version of target
    kind: Deployment        # Target is a Deployment
    name: web-app           # Name of the target
  minReplicas: 2            # Minimum number of pods
  maxReplicas: 10           # Maximum number of pods
  metrics:                  # Metrics to base scaling on
  - type: Resource          # Use resource-based metric
    resource:
      name: cpu             # Metric is CPU utilization
      target:
        type: Utilization   # Target type: Utilization
        averageUtilization: 50 # Desired average: 50% CPU
```

Q14

What is Vertical Pod AutoScaler?

- Purpose: Automatically recommends or updates CPU/memory resource requests & limits for Pods, adjusting “vertically” instead of adding replicas
- Requirement: A component exposing the resource.metrics.k8s.io API (commonly the Metrics Server) must be running so VPA can fetch per-container CPU and memory usage data
- Needs VPA Controllers: VPA Controllers need to be installed

Modes:

- Off: No changes to pods; just stores/suggests recommendations and can be inspected in the VPA object
- Initial: Sets recommended resources only at pod creation. Does not evict or update existing pods
- Recreate: Sets recommended resources when new pods are created and evicts existing pods when the requested resources differ significantly from the new recommendation (respecting the PDBs, if defined)
- Auto(Deprecated): Equivalent to Recreate - will be removed in a future API version
- InPlaceOrRecreate(alpha feature): Assigns resource requests on pod creation as well as updates them on existing pods by leveraging Kubernetes in-place update capability

YAML / Kubernetes

```
# VPA to auto-adjust resources for 'web-app' pods
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-app-vpa          # VPA resource name
spec:
  targetRef:                 # Target to apply VPA on
    apiVersion: apps/v1      # Target API version
    kind: Deployment         # Kind: Deployment
    name: web-app            # Name of the deployment
  updatePolicy:
    updateMode: "Recreate"   # Use explicit mode
                              # instead of deprecated "Auto"
  resourcePolicy:
    containerPolicies:
      - containerName: "*"   # Applies to all containers
        minAllowed:          # Minimum resource limits
          cpu: "100m"        # At least 100 millicores
                              # (0.1 vCPU)
          memory: "128Mi"    # At least 128Mi memory
        maxAllowed:          # Maximum resource limits
          cpu: "1"           # No more than 1 CPU
          memory: "1Gi"      # No more than 1Gi memory
```

Q15

How is HPA different from VPA?

Aspect	HPA
Purpose	Scales the number of pod replicas based on load
Scaling	Mechanism Adds/removes pod replicas to meet demand
Integration	Built-in to Kubernetes by default
Impact on Pods	Creates/destroys pods to handle load
Typical Metrics Used	CPU, memory, or custom metrics
Benefit	Ensures high availability by scaling replicas

Q16

What is Cluster AutoScaler?

- Cluster AutoScaler: Kubernetes component that automatically adjusts the number of nodes in your cluster based on the pending workload.
 - If Pods can't be scheduled due to insufficient resources, it adds nodes
 - If nodes are underutilized and their Pods can be moved elsewhere, it removes nodes
- Integrates with Cloud API: Works directly with your cloud provider's API (e.g., AWS, GCP, Azure) to scale infrastructure (not just Pods)
- How It Works:
 - Scale Up:
 - A new Pod is unschedulable (e.g., not enough memory or CPU)
 - AutoScaler looks for a suitable node group and requests a new node
 - Scale Down:
 - A node is underutilized (low CPU/memory)
 - Pods can be moved to other nodes
 - If safe, the node is drained and removed
- Supported Cloud Providers: Google Cloud (GKE), Amazon Web Services (EKS), Microsoft Azure (AKS), ...

Q17

How does HPA compare to VPA and Cluster AutoScaler?

Feature	HPA	VPA
Scales	Pod replicas	Pod CPU/memory requests
Metrics Used	CPU, memory, custom	Historical usage
Cloud Integration Needed	No	No
Setup Complexity	Low	Medium

Q18

What are some alternatives to Cluster AutoScaler?

Karpenter

- Karpenter is an autoscaler from AWS that quickly adds new nodes to your cluster when needed — designed to replace the older Cluster Autoscaler
 - Faster scaling: Reacts in seconds when Pods can't be scheduled, instead of waiting minutes like Cluster Autoscaler
 - No need for node groups: Karpenter automatically picks the right instance type, so you don't have to manage node pools
 - Cost-efficient with Spot support: It can use cheaper Spot Instances and safely handle interruptions to save money
 - Open and flexible: Though built for AWS (EKS), Karpenter is open source and can be extended to other cloud platforms

Fully managed Kubernetes Managed Services

- Serverless Node Provisioning (Fargate, ACI, GKE Autopilot): Fully managed Kubernetes execution environments provided by cloud platforms that abstract away infrastructure provisioning;
 - AWS Fargate for Amazon EKS and ECS
 - Azure Container Instances (ACI) with AKS virtual nodes
 - GKE Autopilot offering fully-managed Kubernetes clusters without exposing node infrastructure
- Zero Infrastructure Management: Developers can deploy workloads without worrying about node pools, instance types, or scaling policies; the cloud provider dynamically provisions and scales resources to match the workload's needs

CHAPTER 5

Kubernetes Storage

11 Questions & Answers

Q1**Why is persistence important for stateful applications in Kubernetes?**

- Pod Storage is Ephemeral: Data inside a Pod is lost when the Pod is restarted, deleted, or moved to another node
 - Retain Important Data Outside the Pod: Store databases, logs, and user files in persistent storage to avoid data loss
 - Needed for Stateful Apps: Required for apps like MySQL, PostgreSQL, and Kafka that depend on consistent storage
 - Supports Backups and Disaster Recovery: Supports snapshots and recovery strategies to bring back lost or corrupted data
 - Ensures High Availability and Reliability: Keeps data safe during failures

Q2**What is the need for Volumes in Kubernetes?**

- Why Use Kubernetes Volumes?: They allow containers in a Pod to access and share data using a persistent filesystem
 - Persist Container Data: Avoids data loss when containers crash or restart by keeping data outside the container
 - Support Multi-Container Pods: Lets containers within the same Pod share files easily using a common volume
 - Enable Shared Storage Across Pods: Volumes like NFS help share data between Pods, even on different nodes
 - Use Configs and Secrets as Files: Automatically load configuration files or credentials using ConfigMaps and Secrets
 - Provide Temporary Working Space: Use emptyDir volumes for scratch space that lives as long as the Pod does
 - Mount Read-Only Data: Share static content from a container image without modifying it

Q3**Give examples of different types of volumes in Kubernetes**

- **emptyDir**: Temporary volume that provides scratch space for a Pod; data is deleted when the Pod is removed from the node
- **hostPath**: Mounts a file or directory from the host node's filesystem into the Pod
- **nfs**: Mounts a shared NFS (Network File System) volume; allows multiple Pods across nodes to access the same files
- **configMap**: Mount configuration values from key-value pairs directly into the container as files
- **persistentVolumeClaim**: Claim for persistent storage. Kubernetes matches PVC with the right storage!

[YAML](#) / [Kubernetes](#)

```

# Sample Kubernetes Pod using different volume types
apiVersion: v1
kind: Pod
metadata:
  name: volume-demo-pod
spec:
  containers:
    - name: app-container
      image: busybox
      command: [ "sleep", "3600" ]

    # Mount ConfigMap as files
    volumeMounts:
      - name: config-vol
        mountPath: /etc/config

    # Mount temporary scratch space
    - name: temp-vol
      mountPath: /tmp/scratch

    # Mount host path directory
    - name: host-vol
      mountPath: /mnt/host

    # Mount NFS volume
    - name: nfs-vol
      mountPath: /mnt/nfs

    # Mount persistent volume claim
    - name: pvc-vol
      mountPath: /mnt/data

  volumes:
    # 1. configMap volume
    - name: config-vol
      configMap:
        name: app-config
        # ConfigMap must exist in the same namespace

    # 2. emptyDir volume (lives as long as the Pod)
    - name: temp-vol
      emptyDir: {} # Default: node's disk is used

    # Optionally use memory:
    # emptyDir:
    #   medium: Memory

    # 3. hostPath volume
    - name: host-vol
      hostPath:
        path: /var/log
        type: Directory

    # 4. nfs volume
    - name: nfs-vol
      nfs:
        server: 10.0.0.12
        path: /shared-data
        # NFS server must allow client access

    # 5. persistentVolumeClaim
    - name: pvc-vol
      persistentVolumeClaim:
        claimName: data-pvc
        # The PVC must exist before Pod creation

---
# Sample ConfigMap (used above)
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:

```



```
app.properties: |
  mode=production
  max_connections=50

---
# Sample PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: standard
```

Q4

What is Static Volume Provisioning?

- **Define PersistentVolumes Manually:** In static provisioning, cluster admins pre-create PersistentVolumes (PVs) with fixed size, storage type, and access modes
- **Claim Storage with PVCs:** Users create PersistentVolumeClaims (PVCs) to request storage; Kubernetes matches the PVC to a suitable pre-created PV
- **Use Matching Rules for Binding:** Kubernetes binds PVC to PV based on criteria like storage size, access mode, etc.
- **No Automation for Volume Creation:** Unlike dynamic provisioning, the PV must already exist—Kubernetes does not create the volume automatically
- **Suitable for Controlled Environments:** Best used when storage is managed externally (e.g., existing NFS shares) and admins want full control over provisioning

[YAML](#) / [Kubernetes](#)

```

# Static Volume Provisioning Example
# -----
# Admin creates the PersistentVolume (PV) manually
# User creates a PersistentVolumeClaim (PVC)
# Kubernetes binds PVC to a matching PV

# Step 1: PersistentVolume (PV)
apiVersion: v1
kind: PersistentVolume
metadata:
  name: static-pv
spec:
  capacity:
    storage: 1Gi # Size of volume
  accessModes:
    - ReadWriteOnce # One node read/write
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: "/mnt/data"
    type: DirectoryOrCreate
    # For real use, replace with:
    # nfs:
    #   server: 10.0.0.10
    #   path: /shared-nfs-path

# ReclaimPolicy options:
# - Retain: Keep data after PVC deleted
# - Delete: Remove volume with PVC

---

# Step 2: PersistentVolumeClaim (PVC)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: static-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual
  # Must match storageClassName in PV

---

# Step 3: Pod using the PVC
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
spec:
  containers:
    - name: app
      image: busybox
      command: [ "sleep", "3600" ]
      volumeMounts:
        - mountPath: "/usr/share/data"
          name: storage-vol
  volumes:
    - name: storage-vol
      persistentVolumeClaim:
        claimName: static-pvc

```

- Provision Storage Automatically: Dynamic provisioning creates storage volumes on demand when users create PersistentVolumeClaims
 - Avoid Manual Volume Creation: Eliminates the need for admins to manually create PersistentVolumes or interact with storage providers
 - Use StorageClass for Provisioning Rules: Admins define StorageClass objects that specify which provisioner to use and its parameters
 - Offer Multiple Storage Flavors: Allows clusters to support different types of storage (e.g., fast SSD, standard HDD, cloud volumes) using different StorageClass configurations
 - Simplify Storage for Users: Users only need to create a PVC; Kubernetes handles the rest using the defined StorageClass configuration

```

# Dynamic Volume Provisioning Example
# -----
# Admin creates StorageClass once
# Users create PVCs referencing the StorageClass
# Kubernetes dynamically provisions a PV

# Step 1: StorageClass (admin-created using CSI provisioner)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: ebs.csi.aws.com # Recommended driver for AWS EBS
# For other providers:
# - GCP: pd.csi.storage.gke.io
# - Azure: disk.csi.azure.com

parameters:
  type: gp3 # AWS EBS volume type
  fsType: ext4 # Filesystem to use

reclaimPolicy: Delete
# - Retain: keep data after PVC is deleted
# - Delete: auto-remove volume with PVC
# - Recycle: legacy; not supported by most CSI drivers

volumeBindingMode: WaitForFirstConsumer
# - Immediate: bind volume as soon as PVC is created
# - WaitForFirstConsumer: bind only when Pod is scheduled

---

# Step 2: PersistentVolumeClaim (user-created)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: fast-storage
# Must match StorageClass name

---

# Step 3: Pod using the PVC
apiVersion: v1
kind: Pod
metadata:
  name: dynamic-pod
spec:
  containers:
    - name: app
      image: busybox
      command: [ "sleep", "3600" ]
      volumeMounts:
        - mountPath: "/data"
          name: dynamic-vol
  volumes:
    - name: dynamic-vol
      persistentVolumeClaim:
        claimName: dynamic-pvc

```

- Request Storage: Allows users to request specific storage resources without needing to know the underlying storage details
- Works with Both Static and Dynamic Provisioning: PVCs are flexible and can bind to pre-created PersistentVolumes (static) or trigger automatic provisioning (dynamic) via a StorageClass

YAML / Kubernetes

```
# WHAT: Requests persistent storage from the cluster
# WHY: Allows an app to store logs that survive
#       restarts or rescheduling of Pods
# WHEN: Use when data (like logs) must be retained
# WHERE: Defined in the same namespace as the Pod
# HOW: Matches with a compatible StorageClass or PV

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: log-pvc
  # PVC name used by the Pod to reference this storage
spec:
  accessModes:
    - ReadWriteOnce
    # WHAT: Access policy for the volume
    # - ReadWriteOnce: only one node can write
    # - ReadOnlyMany: multiple nodes can read
    # - ReadWriteMany: multiple nodes can read/write

  resources:
    requests:
      storage: 1Gi
      # WHAT: Storage size requested (min guaranteed)
      # WHY: App needs this much space for logs
      # HOW: Cluster finds/provisions matching volume

  storageClassName: fast-ssd
  # WHAT: Tells Kubernetes which StorageClass to use
  # WHY: Enables dynamic provisioning of the volume
  # VARIATION:
  # - Omit this field to use the default StorageClass
  #
  # storageClassName: manual
  # Bind to Pre Created PersistentVolume (Static Provisioning)
  # Ensure storageClassName of PersistentVolume is manual
```

Q7

What is the role of a StorageClass in dynamic provisioning?

- Dynamic Provisioning: Defines how storage is dynamically provisioned, abstracting storage backend details
- Reusability: Enables multiple PVCs to use the same StorageClass for consistent storage provisioning

YAML / Kubernetes

```

# WHAT: Defines a class for dynamic volume creation
# WHY: PVCs use it to auto-provision storage
# WHERE: Cluster-wide resource (not namespaced)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
    # Makes this the default StorageClass
    # PVCs without storageClassName will use this one
    # Only one default StorageClass is allowed per cluster
provisioner: ebs.csi.aws.com
# CSI-based AWS EBS provisioner (recommended)
# For GCP: pd.csi.storage.gke.io
# For Azure: disk.csi.azure.com

parameters:
  type: gp3
  # AWS EBS volume type
  # GCP: pd-standard, pd-ssd
  # Azure: Standard_LRS, Premium_LRS

  fsType: ext4
  # File system to format the volume with
  # Options: ext4 (default), xfs, btrfs, ntfs (depends on OS image)

reclaimPolicy: Delete
# WHAT: Behavior after PVC is deleted
# WHY: To automatically delete the volume and free up resources
# OPTIONS:
# - Retain: keep volume and data after PVC deletion
# - Delete: delete volume (typical for cloud-managed disks)
# - Recycle: deprecated legacy mode (not supported in CSI)

volumeBindingMode: WaitForFirstConsumer
# WHAT: Controls when volume is bound to the node
# WHY: Ensures zone-aware volume placement (especially in multi-zone clusters)
# OPTIONS:
# - Immediate: create volume as soon as PVC is created
# - WaitForFirstConsumer: create volume only after Pod is scheduled (recommended)

```

Q8

PV, PVC and StorageClass - Troubleshooting

Scenario: What happens if a PVC requests more storage than any available PV offers?

- PVC Stays Unbound: Kubernetes can't find a matching PV with sufficient capacity, so the claim remains in Pending state
- Pod Stuck in Pending: If a Pod depends on the PVC, it will also stay in Pending until the claim is resolved
- No Auto-Resize or Retry: Kubernetes doesn't auto-adjust existing PVs
- Fix by Provisioning a Larger PV: Admin must create a compatible PV or enable dynamic provisioning via StorageClass
- Verify with `kubectl describe pvc`: Check for capacity mismatch and related events for troubleshooting

Scenario: You've defined a PVC and deployed a pod using it, but the pod is stuck in Pending. What could be the issue and how do you debug it?

- No Available PV: There may be no static PV available with matching specs, or dynamic provisioning may not be enabled
- Missing StorageClass: If the PVC references a StorageClass that doesn't exist, provisioning fails silently
- Check Events and Describe: Use `kubectl describe pod <pod-name>` and `kubectl describe pvc <pvc-name>` to inspect reasons

Q9

Why would you prefer to use StatefulSet over a Deployment for running a distributed message platform like Kafka?

- Kafka is a Distributed Messaging Platform: Apache Kafka is used for building real-time event-driven systems
- Each Broker Has a Persistent Identity: A Kafka broker is a server that stores data and serves client requests. Each broker in a Kafka cluster has a unique ID and hostname and maintains local storage for topics and partitions.
 - Kafka Brokers Need Stable Identity: Each Kafka broker registers itself with a unique ID and hostname in the cluster
 - Stable DNS Is Critical for Communication: Kafka uses broker hostnames (like `kafka-0.kafka-headless.default.svc.cluster.local`) for internal communication and client access
 - Clients and Brokers Rely on Fixed Hostnames: Without consistent DNS names, clients can't reconnect and brokers can't find each other
- Deployment Fails to Guarantee DNS Stability: Pods created by a Deployment get random names (e.g., `kafka-xyz123`), breaking Kafka's expectations
- StatefulSet Solves These Problems: It gives each Pod a fixed hostname and persistent volume (e.g., `kafka-0` always gets `pv-kafka-0`)
- Ensures Predictable, Reliable Cluster Behavior: Required for stable broker identity, safe storage, and smooth recovery in Kafka deployments

Q10

Can you explain a StatefulSet with an example?

- **Stable Identity:** Assigns each Pod a unique, persistent identity that remains consistent across rescheduling
 - **Persistent Storage:** Ensures each Pod has its own PersistentVolume, maintaining data across restarts
 - **Ordered Deployment:** Deploys and scales Pods in a specific order (pod-0, pod-1, ...), crucial for certain stateful applications
 - **Headless Service:** Often used with headless Services to manage network identities of Pods
- **Each Pod is Named Predictably:** kafka-0, kafka-1, kafka-2
- **Each Pod Gets a Volume:** kafka-storage-kafka-0, etc.
- **Stable DNS with Headless Service:** Required for Kafka to connect broker-to-broker
- **Startup and Shutdown Are Ordered:** Helps Kafka avoid cluster confusion or split-brain

[YAML](#) / [Kubernetes](#)


```
# WHAT: Defines a StatefulSet for a Kafka-like app
# WHY: Each Pod needs a stable name and its own volume
# WHEN: Use when app relies on identity and persistent state
# WHERE: Pods get hostnames like kafka-0, kafka-1, etc.
# HOW: Each Pod gets a PVC and starts in order
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
spec:
  serviceName: kafka-headless
  replicas: 3
  selector:
    matchLabels:
      app: kafka
  template:
    metadata:
      labels:
        app: kafka
    spec:
      containers:
        - name: kafka
          image: bitnami/kafka:latest
          ports:
            - containerPort: 9092
          volumeMounts:
            - name: kafka-storage
              mountPath: /bitnami/kafka
  volumeClaimTemplates:
    - metadata:
        name: kafka-storage
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 10Gi
        storageClassName: fast-ssd
```

```
---
# WHAT: Headless service for stable Pod DNS
# WHY: Allows kafka-0, kafka-1, etc. to resolve by name
# HOW: Used with StatefulSet for stable networking
```

```
apiVersion: v1
kind: Service
metadata:
  name: kafka-headless
spec:
  # Disables allocation of a cluster IP for the service.
  # Creates DNS records for individual Pods
  # Allows direct pod-to-pod communication
  # based on their DNS names
  clusterIP: None
  selector:
    app: kafka
  ports:
    - port: 9092
      name: kafka
```

Q11

How is a Secret different from a ConfigMap, and how are they mounted in a pod?

- Separate Sensitive from Non-Sensitive Data: Use Secrets for passwords, tokens, and keys; use ConfigMaps for configs like URLs and log levels
- Encoded vs Plain Storage: Secrets are base64-encoded and treated as sensitive; ConfigMaps store plain text
- Access-Controlled and Safer: Secrets are stored in etcd with stricter access rules; safer for confidential info
- Same Mounting Options: Both can be mounted as files or exposed as environment variables in Pods
- Example Use Case: Use a Secret for a database password and a ConfigMap for the database hostname in the same Pod

Why are changes to ConfigMaps and Secrets not visible immediately?

- Not Live Reloaded by Default : Updates to ConfigMaps and Secrets are not immediately reflected in running Pods
- No Built-in Watch Mechanism: Kubernetes does not trigger Pod updates automatically on changes to ConfigMaps or Secrets
- Restart Needed for Environment Variables: If used as environment variables, changes only apply after Pod restart
- Volume Mounts Sync with Delay: When mounted as files, updates are polled by kubelet (typically every 1–2 minutes) and not delivered instantly
- Use External Tools for Auto-Restart: Tools like Reloader or Kustomize annotations can automate Pod restarts on changes

```

# WHAT: Stores sensitive data securely
# WHY:  Used for passwords, tokens, keys
# HOW:  Encoded in base64; access-controlled
# WHERE: Created in the Pod's namespace

apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  db-password: cGFzc3dvcmQxMjM=
  # "password123" base64-encoded
  # echo -n "password123" | base64

---
# WHAT: Stores non-sensitive app config
# WHY:  Used for URLs, ports, feature flags
# HOW:  Stored as plain text key-value pairs
# WHERE: Created in the Pod's namespace

apiVersion: v1
kind: ConfigMap
metadata:
  name: db-config
data:
  db-host: my-database.svc.cluster.local
  db-port: "5432"
  db-name: myappdb
  db-user: appuser

---
# WHAT: Pod that uses Secret + ConfigMap
# WHY:  Mounts password + host into container
# WHEN: App reads them from /app/config/
# WHERE: Values mounted as individual files

apiVersion: v1
kind: Pod
metadata:
  name: db-client
spec:
  containers:
    - name: app
      image: busybox
      command:
        - /bin/sh
        - -c
        - >
          echo DB_HOST=$(cat /app/config/db-host);
          echo DB_PASSWORD=$(cat /app/config/db-password);
          sleep 3600;
      volumeMounts:
        - name: db-host-volume
          mountPath: /app/config/db-host
          subPath: db-host
        - name: db-pass-volume
          mountPath: /app/config/db-password
          subPath: db-password
      volumes:
        - name: db-host-volume
          configMap:
            name: db-config
        - name: db-pass-volume
          secret:
            secretName: db-secret

# -----
# VARIATION: Use env instead of file mounts
# env:
# - name: DB_HOST
#   valueFrom:
#     configMapKeyRef:
#       name: db-config
#       key: db-host

```

```
# - name: DB_PASSWORD
#   valueFrom:
#     secretKeyRef:
#       name: db-secret
#       key: db-password
# -----
#
```

CHAPTER 6

Kubernetes Security

10 Questions & Answers

Q1

Why Securing Kubernetes Is Challenging but Critical

Focus Area	Risk or Concern
Cluster Infrastructure	Core components like the API Server, etcd, and Nodes are central control points. If compromised, the whole cluster is at risk.
Application Security	Business apps may have coding flaws or exposed APIs that attackers can exploit.
Container & Host Runtime	Exploiting container runtimes or host OS can lead to container escape (ability of applications or processes running inside a container to access resources outside of the container that are not supposed to be available to them) — compromising the node or adjacent apps.
Pod & Workload Isolation	Pods may have excessive privileges or misconfigured access.
Network Access Control	By default, all Pods can talk to each other.
Secrets & Configuration	Secrets (e.g., passwords, API keys) can be exposed via logs, misconfiguration, or unauthorized access.
Image Supply Chain	Vulnerabilities can be introduced via third-party or outdated container images.
Visibility & Threat Detection	Without monitoring, breaches or misconfigurations may go unnoticed.

Q2

What is the need for Kubernetes Service Accounts?

- **Provide Identity for Pods (Authentication):** Service accounts assign a secure identity to applications running inside Pods so they can interact with the Kubernetes API and external resources
 - **Enable Secure API Access from Pods:** Kubernetes automatically injects a token into each Pod, allowing it to make authenticated API calls using its service account
 - **Limit Access to a Specific Namespace:** Each service account belongs to one namespace, helping isolate permissions and control access locally
 - **Control Permissions with RBAC:** Use Role-Based Access Control to define what actions a service account can perform — such as reading secrets or listing pods
 - **Avoid Using Default Service Account:** For security, define custom service accounts with only the permissions your application truly needs

YAML / Kubernetes

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
  namespace: dev
```

Q3

Practical Example: RBAC with Service Account

- **ServiceAccount Provides Pod Identity:** A ServiceAccount represents an identity for Pods to authenticate with the Kubernetes API
 - **Role Defines Allowed Actions:** A Role lists what actions (like get, list, create) are allowed on which Kubernetes resources within a specific namespace
 - **RoleBinding Connects Identity to Permissions:** A RoleBinding links a Role to a ServiceAccount, granting that identity the defined permissions
 - **Used for Least-Privilege Access:** This trio enables giving Pods only the exact access they need—no more, no less
 - **Supports Multiple Bindings:** One ServiceAccount can be bound to multiple Roles, and a Role can be bound to multiple ServiceAccounts

YAML / Kubernetes

```

# WHAT: ServiceAccount used by the Deployment
# WHY: To provide API identity for the Pod
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pod-reader-sa
  namespace: my-namespace

---
# WHAT: Role that allows listing Pods
# WHY: To grant limited API read access
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: my-namespace
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]

---
# WHAT: Bind Role to ServiceAccount
# WHY: Allow pod-reader-sa to list Pods
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-pod-reader
  namespace: my-namespace
subjects:
- kind: ServiceAccount
  name: pod-reader-sa
  namespace: my-namespace
roleRef:
  kind: Role
  name: pod-reader
  namespace: my-namespace
apiGroup: rbac.authorization.k8s.io

---
# WHAT: Deployment with 1 replica
# WHY: Periodically lists Pods in namespace
# HOW: Uses curl and mounted token
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pod-reader-deploy
  namespace: my-namespace
spec:
  replicas: 1
  selector:
    matchLabels:
      app: pod-reader
  template:
    metadata:
      labels:
        app: pod-reader
    spec:
      # Tells Kubernetes to run this Pod
      # using the specified ServiceAccount
      serviceAccountName: pod-reader-sa
      containers:
      - name: curl
        image: curlimages/curl:latest
        command:
        - /bin/sh
        - -c
        - |
          while true; do
            # Token is automatically mounted at this path by Kubernetes
            TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
            curl -s --header "Authorization: Bearer $TOKEN" \
              --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
              https://kubernetes.default.svc/api/v1/namespaces/my-namespace/pods;
            echo "-----";
            sleep 10;

```


Q4

What is the need for ClusterRole and ClusterRoleBinding?

- **Access Across All Namespaces:** Use ClusterRole when a Pod or user needs to list, get, or modify resources (like Pods or Nodes) across multiple namespaces or the whole cluster
- **Grant Permissions to Non-Namespaced Resources:** Cluster-wide resources like nodes, persistentvolumes, clusterroles, and namespaces can only be managed using a ClusterRole
- **Avoid Over-Provisioning Roles in Each Namespace:** Instead of duplicating the same Role in every namespace, a single ClusterRole simplifies management and ensures consistency
- **Secure But Flexible Access Control:** ClusterRole with ClusterRoleBinding can grant access to specific users, groups, or service accounts across the cluster without modifying every namespace
- **Enable Admin or Read-Only Cluster-Wide Views:** Required for dashboards, monitoring tools (e.g., Prometheus, Lens) that need visibility across namespaces

[YAML / Kubernetes](#)

```

# WHAT: ServiceAccount for pod to use
# WHY: Provides identity to access the API
# WHERE: Namespace 'my-namespace'
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cluster-pod-reader-sa
  namespace: my-namespace

---
# WHAT: Cluster-wide role to read Pods
# WHY: Allows listing all Pods across namespaces
# WHEN: Used by tools needing full visibility
# WHERE: Not namespace-specific (cluster-level)
apiVersion: rbac.authorization.k8s.io/v1

# ClusterRole is required for:
# - Accessing non-namespaced resources (e.g., nodes)
# - Access across all namespaces
#
# Use Role if you only need access
# within a single namespace
kind: ClusterRole
metadata:
  name: cluster-pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]

---
# WHAT: Binds ClusterRole to ServiceAccount
# WHY: Grants service account access to use the role
# WHERE: Applies to my-namespace namespace service account
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: bind-cluster-pod-reader
subjects:
  - kind: ServiceAccount
    name: cluster-pod-reader-sa
    namespace: my-namespace
roleRef:
  kind: ClusterRole
  name: cluster-pod-reader
  apiGroup: rbac.authorization.k8s.io

---
# WHAT: Deployment to list Pods using API
# WHY: Demo to show access via the ServiceAccount
# HOW: Uses curl to call Kubernetes API
# WHERE: Runs in 'my-namespace' namespace
apiVersion: apps/v1
kind: Deployment
metadata:
  name: read-pods-deploy
  namespace: my-namespace
spec:
  replicas: 1
  selector:
    matchLabels:
      app: pod-reader
  template:
    metadata:
      labels:
        app: pod-reader
    spec:
      serviceAccountName: cluster-pod-reader-sa
      containers:
        - name: reader
          image: curlimages/curl:latest
          command:
            - /bin/sh
            - -c
            - >

```

```
while true; do
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token);
curl -s --header "Authorization: Bearer $TOKEN"
--cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
https://kubernetes.default.svc/api/v1/pods;
echo "---";
sleep 10;
done
```

Q5

What is the need for a Network Policy?

- **Enforce Zero Trust Networking:** By default, Kubernetes allows all Pod-to-Pod communication. Network policies help enforce least privilege—only explicitly allowed traffic is permitted
 - **Restrict Unwanted Communication Between Services:** In a microservices setup, not all services should talk to each other. For example, a frontend service may access the backend, but it shouldn't talk directly to the database
 - **Protect Sensitive Data in Transit:** Limit traffic to only expected sources and destinations—e.g., only allow traffic to the database from backend services, not from all Pods
 - **Control Egress and Ingress Traffic:** Ensure that only specific Pods can access external services (egress) or be reached by them (ingress)—important for compliance and audit
 - **Limit Blast Radius of Compromised Pods:** If one Pod is compromised, network policies can prevent it from reaching other sensitive services like payment or admin microservices
 - **Create Logical Security Boundaries:** Apply isolation by namespace, label, or app role (e.g., only role: backend can access role: db)

Q6

Give a Practical Example of Network Policy

[YAML](#) / [Kubernetes](#)

```
# Blocks all incoming traffic unless explicitly allowed
# WHEN: Use as a baseline before applying any allows
# WHY: Ensures "default deny" security posture
# HOW: Apply once for namespace-wide deny
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: my-microservices
spec:
  # Matches all Pods in the namespace
  podSelector: {}
  policyTypes:
    # This policy is about incoming traffic.
    - Ingress
    # No ingress section defined =>
    # no Pods are allowed to receive any traffic
```

```
---
# WHAT: Allow only 'frontend' to talk to 'backend'
# WHY: Enforce service-level isolation
# WHEN: Frontend calls APIs on backend
# WHERE: All in 'my-microservices' namespace
# HOW: Match Pod labels using 'type'
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-from-frontend
  namespace: my-microservices
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
```

```
---
# WHAT: Allow only 'backend' to talk to 'database'
# WHY: Protect DB from direct external access
# WHEN: Backend persists data to database
# WHERE: All in 'my-microservices' namespace
# HOW: Match Pod labels using 'type'
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-database-from-backend
  namespace: my-microservices
spec:
  podSelector:
    matchLabels:
      role: database
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: backend
```

- Pod Security Admission: Enforces Kubernetes' built-in security standards (restricted, baseline, privileged) at the namespace level to control which Pods can be admitted based on their security context
 - Privileged: No restrictions (for legacy workloads)
 - Baseline: Moderate restrictions; avoids known privilege escalations
 - Restricted: Strongest controls; disallows host access, privilege escalation, most volume types
- Fail Early with Clear Errors: Admission controller rejects non-compliant pods at creation time, giving immediate feedback instead of allowing unsafe pods to run
- Enforce Secure Defaults for All Teams: Helps platform teams ensure every developer follows security best practices (e.g., disallow hostPath usage)
- Apply Policies Based on Namespace: Apply different levels of security (restricted, baseline, privileged) to different environments (e.g., prod vs dev)
- Prevent Dangerous Pod Configurations: Blocks pods that attempt to run as root or use privileged containers, reducing the risk of accidental or malicious escalations
- Avoid Misuse of Host Resources: Prevents containers from mounting sensitive host directories or using host networking, which could expose the host node

Q8

How does Pod Security Admission work? What is the role of Pod Security Standards (PSS)?

- **Pod Security Admission:** Pod Security Admission (PSA) is a built-in admission controller that checks Pod specifications before they are created in the cluster
- **Enforce Security Rules at Namespace Level:** PSA works by evaluating Pods against predefined rules configured per namespace
- **Leverage Pod Security Standards (PSS):** Kubernetes defines three built-in security profiles — privileged, baseline, and restricted — to describe increasing levels of security
 - Privileged: No restrictions (for legacy workloads)
 - Baseline: Moderate restrictions; avoids known privilege escalations
 - Restricted: Strongest controls; disallows host access, privilege escalation, most volume types
- **Use Labels to Set Enforcement Mode:** Admins apply labels like `pod-security.kubernetes.io/enforce: restricted` to a namespace to apply a PSS level
- **Offer Flexible Modes of Operation:** PSA supports three modes:
 - `enforce`: Blocks Pods that do not meet the configured Pod Security Standard—used to strictly enforce security policies in production
 - `audit`: Logs violations of the security standard without blocking Pod creation—ideal for monitoring policy compliance
 - `warn`: Displays warnings to users at creation time if a Pod would violate policy—useful for developer awareness during testing or onboarding
- **Apply Different Modes for Dev and Prod:** You can use `warn` in dev namespaces to educate developers, and `enforce` in prod to block insecure Pods
- **Avoid Complex Custom Policies:** PSS gives you a standardized, Kubernetes-native way to enforce common security best practices without writing your own policies

```
# WHAT: Creates a new namespace for secure workloads
# WHY: To isolate and enforce security settings on all Pods
# created in this namespace
apiVersion: v1
kind: Namespace
metadata:
  name: secure-apps
  labels:
    # WHAT: Enforce Pod Security Standard (PSS) level
    # WHY: Blocks Pods that violate this level
    # OPTIONS:
    # - privileged: least secure (allows everything)
    # - baseline: reasonable defaults (e.g. no privileged)
    # - restricted: most secure (e.g. must drop capabilities,
    # run as non-root, no hostPath, etc.)
    pod-security.kubernetes.io/enforce: restricted

    # WHAT: Log violations without blocking the Pod
    # WHY: Helps security teams monitor risk before enforcing
    # OPTIONS: same as 'enforce'
    pod-security.kubernetes.io/audit: restricted

    # WHAT: Warn users during Pod creation if it would violate policy
    # WHY: Educates users early without failing the request
    # OPTIONS: same as 'enforce'
    pod-security.kubernetes.io/warn: restricted

# By adding all three rules:
# You block insecure Pods, log the attempt,
# and show the user a warning—all at once.

# ALTERNATIVE:
# In dev namespaces, you can use:
# pod-security.kubernetes.io/enforce: baseline
# pod-security.kubernetes.io/audit: restricted
# pod-security.kubernetes.io/warn: restricted
# This allows slightly less secure Pods (baseline),
# but still monitors and warns about restricted-level violations.
```

Q9

What is the need for SecurityContext?

- SecurityContext: Defines privilege and access settings for a Pod or container, helping enforce safe defaults
 - Avoid Running as Root: By default, containers may run as the root user, which can be dangerous if compromised
 - Enforce Non-Root Execution: Use `runAsNonRoot` and `runAsUser` to force applications to run with limited privileges
 - Restrict Privilege Escalation: Set `allowPrivilegeEscalation: false` to prevent processes from gaining more permissions than they start with
 - Drop Unnecessary Capabilities: Use `capabilities.drop` to remove Linux kernel capabilities like `NET_ADMIN` (Network Administration) or `SYS_TIME` (Change System Time) for tighter security
 - Apply Consistent Security Defaults: Define `securityContext` at the Pod level to apply secure settings to all containers inside a Pod
 - Improve Isolation and Defense-in-Depth: Helps reduce impact if a container is compromised, protecting host and other workloads

```

# WHAT: A secure Pod definition
# WHY: Enforce strong security settings at both
#       Pod and container levels
# WHEN: Use when running sensitive workloads
# WHERE: Applies cluster-wide unless namespaced
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  # Pod-level securityContext applies to all containers
  securityContext:
    runAsUser: 1000
    # WHAT: Run processes as user ID 1000
    # WHY: Prevent root-level access in containers
    # WHEN: App runs safely as non-root

    fsGroup: 2000
    # WHAT: Shared file system group ID
    # WHY: Enables shared volume access across
    #       all containers in the Pod
    # WHEN: Needed when volumes must be writable
    #       by multiple containers

  containers:
    - name: app
      image: myapp:latest
      # Container-specific security settings
      securityContext:
        allowPrivilegeEscalation: false
        # WHAT: Prevents processes inside the container
        #       from gaining more privileges than their parent
        #       (e.g., using setuid/setgid to become root)
        # WHY: Blocks privilege escalation attacks

        readOnlyRootFilesystem: true
        # WHAT: Makes container's root filesystem read-only
        # WHY: Prevents tampering or writing temp files
        # WHEN: App doesn't need to write to root FS
        # NOTE: Use tmp or volume mounts if write is needed

      capabilities:
        drop:
          - ALL
        # WHAT: Drop all Linux capabilities
        # WHY: Follows least-privilege principle
        # WHEN: App doesn't require special system calls

```

Q10

List a few tools to enhance security posture of workloads running in Kubernetes

Tool	Category
Trivy	Vulnerability Scanning
Snyk	Vulnerability Scanning
Clair	Vulnerability Scanning
Falco	Runtime Threat Detection
Seccomp	Runtime Hardening
AppArmor	Runtime Hardening
SELinux	Host & Container Isolation

CHAPTER 7

Kubernetes Scheduling

13 Questions & Answers

Q1

What is the need for Node Affinity?

- **Control Where Pods Run:** Sometimes, you want Pods to run only on specific nodes (e.g., nodes with GPUs, SSDs, or located in a particular zone)
- **Use Node Affinity for Targeted Scheduling:** Node Affinity lets you influence which nodes a Pod can be scheduled on, based on node labels
- **Ensure Resource Compatibility:** Schedule Pods on nodes that meet special hardware or configuration needs, like GPU nodes for ML workloads
- **Enhance Performance and Isolation:** Co-locate Pods that benefit from shared cache or avoid placing high-traffic Pods on same node for better performance
- **Improve Fault Tolerance:** Run workloads on nodes in separate locations to reduce the impact of failures in one area
- **Avoid Overloading Critical Nodes:** Keep non-critical apps away from nodes reserved for sensitive or resource-heavy workloads

Q2

Practical Example: Node Affinity[YAML / Kubernetes](#)

```

# WHAT: Pod with Node Affinity Rules
# WHY: Ensure this Pod is scheduled only on nodes
#       with specific labels (e.g., node-type=high-mem)
# WHEN: Needed when workloads require specialized
#       hardware, OS, or zone-specific nodes

apiVersion: v1
kind: Pod
metadata:
  name: memory-intensive-app
spec:
  containers:
    - name: app
      image: nginx
      resources:
        requests:
          memory: "512Mi"
          cpu: "250m"
      # Simple demo container; replace as needed

  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        # WHAT: Hard requirement for node selection
        # WHY: Pod will only be scheduled if the
        #       rule matches at scheduling time
        nodeSelectorTerms:
          - matchExpressions:
              - key: node-type
                operator: In
                values:
                  - high-mem
            # WHAT: Match nodes labeled with
            #       node-type=high-mem
            # WHY: Needed for workloads that
            #       require large memory capacity

        # ALTERNATIVE: Use 'preferred' affinity
        # Allows more flexible scheduling
        #
        # preferredDuringSchedulingIgnoredDuringExecution:
        #   - weight: 1
        #     preference:
        #       matchExpressions:
        #         - key: node-type
        #           operator: In
        #           values:
        #             - high-mem
        #   # WHAT: Try to schedule on high-mem nodes
        #   # WHY: Helps when strict placement isn't required
        #   # WHEN: Use when you prefer a node but can fall back

# TO MAKE THIS WORK:
# Label your node using:
# kubectl label nodes <node-name> node-type=high-mem

```

Q3

What is the difference between Node Affinity and Node Selector?

- **Simpler and Older Version: Node Label Selector** is Simpler and Older version of Node Affinity
 - **Straightforward Syntax:** Easy to understand and implement for basic use cases
 - **Limited Flexibility:** Only supports exact matches with one or more key-value pairs — Cannot define complex conditions with advanced operators (In, NotIn)
- **Node Affinity is Recommended for Modern Use:** Node Affinity is Fully backward-compatible with Node Selector logic and is more flexible

Code

```
# Node Selector - OLD (Basic)
nodeSelector:
  disktype: ssd

# Node Affinity - NEW and RECOMMENDED
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: disktype
              operator: In # Also supports NotIn, Exists, etc.
              values:
                - ssd
                - another-value
```

Q4 Why do we need Pod Affinity?

- **Control Pod Co-Location:** Sometimes you want certain Pods to be scheduled on the same node as others — for better performance or tighter integration
 - **Use Pod Affinity to Group Related Pods:** Ensures Pods with similar roles (e.g., app and sidecar, or tightly coupled microservices) are placed together
 - **Reduce Latency Between Services:** Co-locating Pods improves communication speed when services need to frequently talk to each other
 - **Support Shared Resource Usage:** Helps when Pods share a persistent volume (e.g., via ReadWriteMany) or access the same local cache or hardware
 - **Improve Data Locality and Throughput:** Useful in big data or AI workloads where data needs to stay close to compute for fast access
 - **Apply Topology-Aware Scheduling:** Pod affinity can be used across zones, or nodes — by specifying the topologyKey

Q5 Practical Example: Pod Affinity

YAML / Kubernetes

```

# WHAT: Pod with Pod Affinity
# WHY:  Schedule this Pod close to another Pod
#       that has specific labels (e.g., app=backend)
# WHEN: Needed for performance or locality reasons—
#       for example, frontend near backend
# WHERE: Works within same topology (e.g., zone or node)

apiVersion: v1
kind: Pod
metadata:
  name: frontend-app
spec:
  containers:
    - name: app
      image: nginx
      # Simple container for demo; replace as needed

  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        # WHAT: Hard rule for scheduling
        # WHY: Pod will only be scheduled if another
        #       matching Pod exists in the same zone
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - backend
              # WHAT: Match any Pod with app=backend
            topologyKey: "kubernetes.io/hostname"
          # WHAT: Match must be in same node
          # OPTIONS:
          # - kubernetes.io/hostname: same node
          # - topology.kubernetes.io/zone: same zone
          # - topology.kubernetes.io/region: same region

        # ALTERNATIVE: Use preferred affinity
        # This allows scheduling elsewhere if match fails
        #
        # preferredDuringSchedulingIgnoredDuringExecution:
        #   - weight: 100
        #     podAffinityTerm:
        #       labelSelector:
        #         matchExpressions:
        #           - key: app
        #             operator: In
        #             values:
        #               - backend
        #       topologyKey: "kubernetes.io/hostname"
        #     # WHAT: Prefer placing near 'backend' Pods
        #     # WHY: Useful when co-location is desirable
        #     # WHEN: Use when strict placement isn't required

# TO MAKE THIS WORK:
# Make sure another Pod exists with:
# labels:
#   app: backend

```

Q6

What is Anti-Affinity?

- **Avoid Placing Similar Pods Together:** Anti-affinity ensures that certain Pods are not scheduled on the same node (or zone) as other matching Pods
- **Define Rules Using Labels and Topology:** Anti-affinity is label-based and supports topologyKey (like `kubernetes.io/hostname` or `topology.kubernetes.io/zone`) to control the scope of separation
- **Improve High Availability:** By spreading replicas across nodes or zones, anti-affinity helps prevent all replicas from going down due to a single node or zone failure
- **Prevent Single Point of Failure:** Especially useful for replicated workloads (like `StatefulSets`) where all Pods shouldn't fail together
- **Minimize Resource Contention:** Keeps resource-hungry Pods apart so they don't compete for CPU, memory, or disk on the same node

Q7

Practical Example: Anti Affinity

YAML / Kubernetes

```
# WHAT: Pod with Pod Anti-Affinity
# WHY:  Ensure this Pod does NOT run on the same node
#       as any Pod with label app=backend
# WHEN: Used to spread workloads for availability,
#       performance, or compliance
# WHERE: Works within defined topology (e.g., node)

apiVersion: v1
kind: Pod
metadata:
  name: analytics-app
spec:
  containers:
    - name: analytics
      image: busybox
      command: ["sleep", "3600"]
      # Dummy workload to hold the Pod

  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        # WHAT: Hard rule to avoid specific co-location
        # WHY: Prevent running alongside backend Pods
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - backend
            # WHAT: Match Pods labeled app=backend
          topologyKey: "kubernetes.io/hostname"
          # WHAT: Enforce on same node level
          # OPTIONS:
          # - kubernetes.io/hostname: different nodes
          # - topology.kubernetes.io/zone: different zones

# TO TEST:
# Schedule another Pod with:
# labels:
#   app: backend
```

Q8

Node Affinity vs Pod Affinity vs Pod Anti-Affinity

Feature	Description
Node Affinity	Schedules pods on nodes based on specific node labels and conditions
Pod Affinity	Ensures pods are scheduled close to other related pods (same node, zone or region)
Pod Anti-Affinity	Prevents pods from being scheduled nearby other specific pods

Q9

What are Taints and Tolerations?

- Prevent Pods from Running on Unsuitable Nodes: Taints let you mark a node as unsuitable for general Pods unless they explicitly tolerate it
- Use Taints to Repel Pods: Nodes can be tainted to repel Pods that don't match
- Allow Specific Pods Using Tolerations: Pods add tolerations that match taints, signaling they're allowed to run on those nodes

Taint = Node with "Only Certified Workloads Allowed" Sign

Toleration = Pod with "Certified Workloads" Badge

- Isolate Workloads Based on Node Roles: Run sensitive or special workloads (like GPU jobs or system daemons) only on tainted nodes, avoiding interference

- Control Scheduling Behavior Precisely: Combine taints and tolerations to enforce rules such as:

Don't schedule general Pods on GPU nodes (NoSchedule)

Prefer not to run Pods on preemptible nodes (PreferNoSchedule)

Evict Pods if a taint is added at runtime (NoExecute)

- DO YOU KNOW?: Kubernetes taints master/control-plane nodes to prevent regular workloads (Pods) from running on them

kubectl describe node <master-node-name> -> Taints: node-role.kubernetes.io/master:NoSchedule ("Don't schedule Pods here unless they tolerate this taint.")

Q10

Practical Example: Taints and Tolerations

YAML / Kubernetes


```

# -----
# STEP 1: Taint the Node (done separately via CLI)
#
# Command:
# kubectl taint nodes node-1 teamA=true:NoSchedule
#
# WHAT: Adds a taint to 'node-1'
# WHY: Prevent Pods from scheduling unless they tolerate it
# FORMAT: key=value:Effect
# EFFECTS:
# - NoSchedule: Don't schedule unless tolerated
# - PreferNoSchedule: Try to avoid if possible
# - NoExecute: Evict existing Pods that don't tolerate
# -----

# -----
# STEP 2: Pod with Matching Toleration
# -----

apiVersion: v1
kind: Pod
metadata:
  name: team-a-app
spec:
  containers:
    - name: nginx
      image: nginx
      # Simple web server container

  tolerations:
    - key: "teamA"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"
      # WHAT: Allows scheduling onto nodes with
      #       taint 'teamA=true:NoSchedule'
      # WHY: Pod explicitly says "I tolerate this taint"
      # WHEN: Node is tainted for team-specific workloads

# -----
# VARIATION: Tolerate Any Taint with Specific Effect
#
# tolerations:
# - operator: "Exists"
#   effect: "NoSchedule"
#
# WHAT: Tolerates any taint with effect 'NoSchedule'
# WHY: Used when key/value don't matter
# -----

# Master Node
# Taints: node-role.kubernetes.io/master:NoSchedule
#
# To Run something on Master Node
# Add a toleration in the spec
# tolerations:
# - key: "node-role.kubernetes.io/master"
#   operator: "Exists"
#   effect: "NoSchedule"

```

- Prevent Too Many Pods from Going Down at Once: Pod Disruption Budgets (PDBs) ensure a minimum number of replicas stay available during voluntary disruptions
 - Handle Planned Maintenance Safely: During node upgrades, autoscaling, or draining, PDBs prevent all replicas from being evicted at the same time
 - Maintain Application Availability: Keep enough Pods running to serve traffic
 - Protect Stateful and Critical Apps: Use PDBs for databases, APIs, or services that require high availability — even during cluster changes
 - Avoid Downtime in Rolling Updates: Kubernetes respects PDBs during deployments and avoids deleting too many Pods at once

YAML / Kubernetes

```
# WHAT: Pod Disruption Budget (PDB)
# WHY: Ensures availability during voluntary
#       disruptions (e.g., node upgrades)
# WHEN: Use for critical apps needing high uptime

apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 1
  # WHAT: Keep at least 1 Pod always running
  # WHY: Prevent full downtime during drains
  # OPTIONS:
  # - minAvailable: count or %
  # - maxUnavailable: count or %
  # NOTE: Use only one at a time

  selector:
    matchLabels:
      app: my-app
      # WHAT: Target Pods with this label
      # WHY: Apply PDB only to specific Pods

# -----
# VARIATION: Use maxUnavailable instead
#
# spec:
#   maxUnavailable: 1
#
# WHAT: Allow disruption of 1 Pod at a time
# -----
```

Q12

Cordon vs Drain

	Cordon
Function	Marks a node as unschedulable; prevents new Pods from being scheduled
Pod Removal	Does not remove existing pods; they continue running
Use Case	Stop new workloads without disrupting running ones
Effect on Node	Node remains active and continues serving current Pods
Command	<code>kubectcl cordon <node-name></code>

Q13

Debugging Problems with Starting Up Pods

- **No Pod is Running:** Pod is not created or crashes immediately due to issues in its definition or during startup
 - Focus on Pod Configuration First!
 - (COMMON ISSUE) Something is wrong in Pod definition or startup: Check for YAML errors, invalid image names, missing configs, volume mount failures, or init container errors
 - Use Diagnostic Commands: Use `kubectcl get pods`, `kubectcl describe pod`, and `kubectcl logs` to identify startup issues
- **Some Pods Running, Others Not:** Some Pods are scheduled and running, but others are stuck in Pending state
 - (COMMON ISSUE) Pod scheduling is blocked due to cluster/node constraints: Issues with resources, taints, affinity, node selectors, or Anti-Affinity can prevent scheduling
 - Use Scheduling Diagnostics: Use `kubectcl describe pod`, `kubectcl describe node`, and `kubectcl get nodes --show-labels` to debug scheduling issues

CHAPTER 8

Kubernetes Observability

9 Questions & Answers

Q1

What is Monitoring?

- **Monitoring:** Monitoring helps detect if a Pod, Node, or application crashes or becomes unresponsive, enabling faster recovery
 - **Identify and Resolve Issues Quickly:** Real-time alerts help detect anomalies like memory leaks, traffic spikes, or API failures before users are affected
 - **Track Resource Usage Over Time:** Tools like Prometheus collect CPU, memory, and disk usage data to optimize resource allocation and reduce cost
 - **Visualize Cluster Performance Trends:** Dashboards (e.g., Grafana) provide easy-to-understand visuals for cluster-wide behavior and app performance
 - **Support Auto-Scaling Decisions:** Horizontal Pod Autoscalers (HPA) rely on metrics (like CPU usage) to decide when to add or remove pods
 - **Aid in Capacity Planning:** Historical monitoring data helps plan for future growth and avoid bottlenecks
 - **Strengthen Security Posture:** Alerting on unusual patterns (e.g., sudden spike in traffic) can surface potential security breaches or attacks

Q2

What is Observability?

- **Observability:** Observability lets you ask questions about your system's current state without having to pre-define every scenario
 - **Go Beyond Monitoring:** While monitoring alerts on known issues, observability helps investigate unknown or unexpected problems
 - **Correlate Metrics, Logs, and Traces:** Combines performance data (metrics), detailed events (logs), and request flows (traces) to give a full picture of what's happening
 - **Diagnose Complex Failures Faster:** Helps identify root causes across distributed systems by following request paths and spotting anomalies
 - **Improve Reliability and User Experience:** Enables teams to detect, debug, and resolve issues before users are impacted
 - **Enable Proactive Operations:** Allows engineers to explore trends and detect degraded performance early—even before alerts are triggered
 - **Essential for Microservices and Cloud-Native Apps:** With many moving parts, observability is critical for understanding interactions and performance across services

Q3**Monitoring vs Observability**

Aspect	Monitoring
Definition	Tracking known metrics and events against predefined thresholds
Goal	Detect and alert on known problems or symptoms
Data Sources	Primarily metrics (CPU, memory, request rate, error rate)
Focus	Alert on predefined conditions (e.g., CPU > 80%)
Scope	Point-in-time status of specific components or metrics
Mindset	Known-knowns: What we expect to happen and want to monitor
Example	Alert me if memory usage > 90% on node X

Q4**What is OpenTelemetry?**

- **Understanding Need for Consistent Observability Standards:** Modern apps span multiple services, languages, and environments—making it hard to trace performance, debug issues, and understand system behavior using isolated tools
 - **What is OpenTelemetry:** An open-source, vendor-neutral standard for collecting metrics, logs, and traces from applications
 - **Collects Telemetry Data in One Format:** Unifies different types of data across services, so you don't have to use multiple tools or agents for different data types
 - **Supports Multiple Programming Languages:** Enables consistent observability whether your microservices are in Java, Python, Go, or Node.js
 - **Sends Data to Any Backend:** Export telemetry to systems like Prometheus, Jaeger, Grafana, Datadog, or Elastic without vendor lock-in
 - **Enables End-to-End Visibility:** Helps correlate latency spikes, failed requests, or memory usage with specific services or traces across the entire system
 - **Simplifies Debugging and Root-Cause Analysis:** Makes it easier to find where and why failures happen—even in complex distributed environments
 - **Backed by CNCF and Major Cloud Providers:** Ensures it's a trusted, long-term standard with active community and enterprise support

Q5

How well is OpenTelemetry supported in Kubernetes?

- **Designed for Cloud-Native Environments:** OpenTelemetry is designed to work seamlessly with platforms like Kubernetes
- **Supports All Telemetry Types:** Collects metrics, logs, and traces from Pods, Nodes, and Services
- **Kubernetes Metadata Enrichment:** Automatically adds Pod names, Namespaces, Node info, and Labels to telemetry data, making debugging easier
- **Collector Works as a DaemonSet or Sidecar:** The OpenTelemetry Collector can run as:
 - A Sidecar to collect application-specific telemetry
 - A DaemonSet to collect node-level and container-level metrics
- **Supports Export Transformations:** The Collector allows you to filter, batch, transform, and export data to one or more backends
- **Integration with Popular Tools:** Easily integrates with Prometheus, Grafana, Jaeger, Zipkin, Datadog, Elastic, etc., for backend analysis

Q6

How does a Service Mesh Work?

- **Inject Sidecar Proxies into Pods:** Each service Pod runs alongside a sidecar proxy (like Envoy) that intercepts all incoming and outgoing traffic
- **Control Plane Configures the Mesh:** A central control plane (e.g., Istio, Linkerd) manages policies, security settings, and traffic rules across proxies
- **Inter-Service Communication Goes Through Proxies:** Instead of talking directly, services route requests through their sidecars, which handle encryption, retries, and load balancing
- **Observability Data Is Collected Automatically:** Proxies capture metrics, logs, and traces without changes to application code, enabling rich observability
- **Security Is Enforced Transparently:** Sidecars use mTLS to verify identities and encrypt data in transit between services

Q7

List a Few Observability Tools in Kubernetes

Tool	Category
OpenTelemetry	Telemetry Instrumentation
Istio	Service Mesh + Telemetry
Prometheus	Metrics
Kube-State-Metrics	Cluster State Metrics
Grafana	Visualization & Dashboards
Jaeger	Distributed Tracing
Grafana Tempo	Distributed Tracing
Elasticsearch	Log Storage & Search
Kibana	Log Visualization

Q8

What is need for Probes?

- **Probes:** Probes help Kubernetes know if your application is working correctly and ready to serve traffic
- **Enable Smooth Rolling Updates:** Probes ensure new Pods are healthy before terminating old ones, reducing downtime during deployments

Probe Type	Purpose
Startup Probe	Checks if the container application has started up
Liveness Probe	Checks if the container is alive and running properly
Readiness Probe	Checks if the container is ready to serve requests (Sometimes, applications are temporarily unable to serve traffic. In such cases, you don't want to kill the application, but you don't want to send it requests either.)

[YAML / Kubernetes](#)

```

apiVersion: v1
kind: Pod
metadata:
  name: app-with-probes
  labels:
    app: probe-demo
spec:
  containers:
    - name: my-app
      image: nginx
      ports:
        - containerPort: 80

# WHAT: Startup Probe
# WHY: Allows app more time to start
# WHEN: Blocks liveness and readiness checks until startup passes
startupProbe:
  httpGet:
    path: /
    port: 80
    # allow 30 consecutive failures before restarting the container
    failureThreshold: 30
    # Run every 5 seconds
    periodSeconds: 5

# WHAT: Readiness Probe
# WHY: Checks if app is ready to receive traffic
# WHEN: Only sends traffic after this passes
readinessProbe:
  httpGet:
    path: /ready
    port: 80
    # Waits 5 seconds after container starts
    # before performing the first probe
    initialDelaySeconds: 5
    # How often to execute
    periodSeconds: 5

# WHAT: Liveness Probe
# WHY: Checks if the app is alive
# WHEN: If it fails repeatedly, Pod is restarted
livenessProbe:
  httpGet:
    path: /healthy
    port: 80
    # Wait 10 seconds after the container starts
    # before performing the first health check
    initialDelaySeconds: 10
    # How often to execute
    periodSeconds: 10
    # Number of consecutive failures
    # before marking unhealthy
    failureThreshold: 3

# NOTE:
# You can use `exec` instead of `httpGet`:
#
# livenessProbe:
#   exec:
#     command:
#       - cat
#       - /tmp/healthy
#

```

- **Deploy One Pod Per Node Automatically:** A DaemonSet ensures that a copy of a Pod runs on every Node (or selected Nodes) in the cluster without manual intervention
- **Collect Logs and Metrics Consistently:** Commonly used to run system and application observability pods on every node in the cluster to enable centralized observability
- **Enable Node-Level Monitoring and Security:** Ideal for running node-level pods like security agents, or storage drivers
- **Control Pod Placement Using Node Selectors or Tolerations:** Can be configured to run only on specific node types (e.g., GPU nodes, tainted nodes)

YAML / Kubernetes

```
# WHAT: Run a Pod on every Node in the cluster
# WHY: For node-level tasks like monitoring/logging
# WHEN: You need to ensure the same Pod runs
#       on all current and future Nodes
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-monitor
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app: node-monitor
    # WHAT: Links the DaemonSet to Pods it manages

  template:
    metadata:
      labels:
        app: node-monitor
    spec:
      containers:
        - name: monitor-agent
          image: busybox
          command:
            - /bin/sh
            - -c
            - >
              while true; do
                echo "Monitoring node...";
                sleep 60;
              done
            # WHAT: Basic infinite loop for demo
            # WHY: Represents a monitoring process

# -----
# VARIATION: Run only on specific Nodes
# Add nodeSelector to limit to node types
#
# spec:
#   template:
#     spec:
#       nodeSelector:
#         disktype: ssd
#
# WHY: Useful if only some nodes need this agent
# -----
```

CHAPTER 9

More Kubernetes Essentials

11 Questions & Answers

Q1

What is CRI?

- Understanding the Need for CRI (Container Runtime Interface): Kubernetes needs to run containers, but there are many container runtimes (Docker, containerd, CRI-O). Kubernetes needed a consistent way to talk to any container runtime.
- What is CRI: CRI is a standard interface that allows Kubernetes to communicate with different container runtimes in a pluggable way, without being tightly coupled to one specific runtime.
- (Use Case) Enable Runtime Flexibility: Lets Kubernetes switch between container runtimes (like Docker to containerd) without changing Kubernetes components
- (Advantage) Encourages Ecosystem Innovation: CRI allows runtime developers to innovate independently while still being Kubernetes-compatible
- Most Popular CRI Implementations:
 - containerd: A lightweight, industry-standard container runtime created by Docker and donated to CNCF; it is now the default runtime in many Kubernetes distributions like GKE and EKS
 - CRI-O: A Kubernetes-native runtime developed by the CNCF community to support Open Container Initiative (OCI) standards; used in OpenShift and other enterprise Kubernetes platforms
 - Docker (via dockershim): Previously used in Kubernetes but deprecated since version 1.20; Docker required a compatibility shim (dockershim) because it didn't implement CRI natively (A shim is a small software layer that connects Kubernetes to a container runtime that does not natively support the Container Runtime Interface)

Q2

What is CNI?

- **Networking:** Network plugins provide essential features for Kubernetes clusters:

Assign IP to Each Pod: Every Pod gets a unique IP so it can communicate like a real machine

Enable Pod-to-Pod Communication: Supports communication between Pods, even if they're on different Nodes

Support Service Discovery: Works with CoreDNS and kube-proxy to route traffic to the right Pod using service names

Allow External Access: Enables outbound internet access and supports exposing Pods using LoadBalancers, NodePorts, or Ingress

Control Network Policies: Implements rules that define which Pods can talk to which other Pods

- **CNI: Container Network Interface** — a standard interface between container runtimes and network plugins

Implementations: Flannel, Calico, Cilium, Weave, etc

Q3

What is CSI?

- **Understanding Need For Persistent Storage:** Data inside a Pod is lost when the Pod is restarted, deleted, or moved to another node

- **What is Persistent Storage:** Allows you to retain databases, logs, and user files in persistent storage to avoid data loss

- **Needed for Stateful Apps:** Required for apps like MySQL, PostgreSQL, and Kafka

- **What is CSI (Container Storage Interface):** A standard plugin interface that enables Kubernetes to work with different storage systems (cloud or on-prem) in a consistent and pluggable way

- **Decouples Storage from Kubernetes Core:** Keeps Kubernetes lean and modular by offloading storage integration to CSI-compliant drivers

- **Integrate External Storage Systems Easily:** Works with AWS EBS, GCP PD, Azure Disk, NFS, etc. — no need to modify Kubernetes core

AWS EBS CSI Driver: Manages Amazon EBS volumes for Kubernetes

AWS EFS CSI Driver: Manages AWS Elastic File System for Kubernetes

Azure Disk CSI Driver: Manages Azure Disk volumes for Kubernetes

GCP Persistent Disk CSI Driver: Manages Google Compute Engine Persistent Disk volumes

NFS CSI Driver: Provides NFS storage to Kubernetes

- **Enables Backups and Restore:** Supports snapshots, cloning, and volume resizing

Q4

How do you setup Highly available Kubernetes cluster?

- **Prevent Single Point of Failure:** A highly available (HA) Kubernetes setup ensures the cluster stays functional even if some nodes or components fail
 - **Use Multiple Control Plane Nodes:** Deploy 3 or more master nodes (API Server, Controller Manager, Scheduler) across availability zones
 - **Configure an External Load Balancer:** Direct traffic to healthy API servers across master nodes using an external load balancer
 - **Deploy ETCD in HA Mode:** Use an odd number (usually 3 or 5) of etcd members
 - **Distribute Worker Nodes Across Zones:** Spread worker nodes across failure domains to maintain app availability during node failures
 - **Enable Persistent Volumes in Multiple Zones:** Ensure storage classes use multi-zone or regional disks for resilient storage
 - **Secure Communication Between Components:** Use TLS certificates, RBAC, and encryption at rest for reliable and secure communication
 - **Monitor and Backup Regularly:** Use observability tools and take regular etcd backups to recover quickly from outages or data loss

Q5

List Some Important Controllers in Kubernetes

- **Node Controller:** Detects and responds when nodes go down or become unreachable
- **ReplicaSet:** Maintains a stable set of replica Pods running at all times
- **Deployment Controller:** Manages rolling updates and rollbacks
- **StatefulSet Controller:** Manages stateful applications with stable network identities and persistent storage
- **DaemonSet Controller:** Ensures a copy of a Pod runs on all (or selected) nodes
- **Service Controller:** Manages service objects and configures load balancers
- **Horizontal Pod AutoScaler Controller:** Automatically adjusts the number of Pods based on CPU/memory usage or custom metrics
- **Namespace Controller:** Manages cleanup and lifecycle of Kubernetes namespaces
- **Ingress Controller:** Manages external HTTP(S) traffic and routes it to services based on rules (though not part of core K8s, it's a commonly used controller)

Q6

What is a Operator?

- **Understanding Need for Application Lifecycle Automation:** Kubernetes manages deployments and scaling, but struggles with day-2 operations like backups, version upgrades, and recovery for complex applications like databases (MongoDB, for example) or messages systems (Kafka, for example)
- **What are Kubernetes Operators:** Kubernetes Operators extend the platform by automating the full lifecycle of an application — install, upgrade, failure recovery, and more
- **Bring Human Operational Knowledge into Kubernetes:** Just like a human operator knows how to upgrade or recover a database, a Kubernetes Operator does the same using code
- **Manage Stateful Applications:** Operators are perfect for managing apps that need persistent storage, or other special features (e.g., Kafka, MongoDB)
- **Automate Upgrades and Patching:** Helps apply rolling updates or version upgrades without downtime
- **Simplify Backups and Disaster Recovery:** Automatically schedule backups and restore from backups when failures happen
- **Popular Real-World Operators:**
 - Prometheus Operator – for monitoring stack
 - MongoDB Operator – for automated DB provisioning
 - Kafka Operator – for Kafka clusters in Kubernetes
- **Example: Setting up MongoDB using MongoDB Operator in Kubernetes**
 - Step 1: Install the MongoDB Kubernetes Operator (`kubectl apply -k github.com/mongodb/mongodb-kubernetes-operator/config/default`)
 - Step 2: Create a namespace (`kubectl create namespace mongodb`)
 - Step 3: Deploy a MongoDB Custom Resource using Custom Resource Definition (CRD) (`kubectl apply -f mongodb.yaml -n mongodb`)

YAML / Kubernetes

```
apiVersion: mongodbcommunity.mongodb.com/v1
kind: MongoDBCommunity
metadata:
  name: my-mongodb
  namespace: mongodb
spec:
  members: 3
  type: ReplicaSet
  version: "6.0.6"

# AND A LOT OF OTHER DETAILS FOLLOW!
```

Q7

Controllers vs Operators

Aspect	Controllers
Purpose	Controllers in Kubernetes continuously monitor and adjust built-in resources like pods and deployments to match the desired state
What They Manage	Native Kubernetes resources like Pods and Deployments
How They Are Built	Built-in to Kubernetes platform
Level of Intelligence	Simple reconciliation: create/update/delete pods
Use of Custom Resources	Do not require Custom Resource Definitions
Examples	ReplicaSet & Deployment Controller

Q8

What is the need for Helm?

- Understanding Need For Kubernetes Package Management: Managing complex applications with many YAML files (Deployments, Services, ConfigMaps, Secrets) is error-prone
 - What is Helm: A package manager for Kubernetes that bundles multiple related YAML files into a single reusable unit called a Helm chart
 - Simplifies Installation and Upgrades: Lets you install or upgrade an entire app (like MySQL or Prometheus) with a single command
 - Promotes Reusability and Standardization: Helm charts can be reused across dev, QA, and production environments — just change a few values
 - Reduces Duplication with Templates: Helm uses templates and variable substitution to avoid copying the same YAML again and again
 - Widely Adopted for Ecosystem Apps: Most popular tools in the cloud-native world (like ArgoCD, Grafana, Redis) offer official Helm charts for quick and consistent installs

[YAML / Kubernetes](#)

```

# WHAT: Example Helm Template for a Kubernetes Deployment
# HOW: Variables defined in values.yaml are injected into templates

# FILE: templates/deployment.yaml
# This file defines the Kubernetes Deployment
# using Helm template expressions like {{ .Values.* }}

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.app.name }}
  labels:
    app: {{ .Values.app.name }}
spec:
  replicas: {{ .Values.app.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Values.app.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.app.name }}
    spec:
      containers:
        - name: {{ .Values.app.name }}
          image: "{{ .Values.app.image }}:{{ .Values.app.tag }}"
          ports:
            - containerPort: {{ .Values.app.port }}
          resources:
            limits:
              memory: {{ .Values.app.resources.memory }}
              cpu: {{ .Values.app.resources.cpu }}

---

# FILE: values.yaml
# This file provides values injected into the template
# Default values can be overridden at install time
app:
  name: myapp
  image: nginx
  tag: latest
  port: 80
  replicaCount: 2
  resources:
    memory: "128Mi"
    cpu: "500m"

---

# FILE: Chart.yaml
# Helm metadata file describing this chart
apiVersion: v2
name: myapp
description: A simple Helm chart for Kubernetes
type: application
version: 0.1.0
appVersion: "1.0"

# NOTE:
# You install this chart using:
# helm install myapp ./mychart
#
# You can override values with:
# helm install myapp ./mychart -f custom-values.yaml

```

- Helm can be complex: Creating charts with templating could be too complex a task for simple applications
 - BUT Customizing Kubernetes YAMLs is Needed: Repeating YAML files across environments (dev, qa, staging, prod) leads to duplication and errors
 - What is Kustomize: Kustomize is a built-in Kubernetes tool that lets you customize YAMLs without modifying the original files — using overlays, patches, and configuration layers
 - Simplifies Environment-Specific Deployments: Lets you reuse a base configuration and apply changes for different environments (e.g., different replica counts, image tags, config maps)
 - Avoids Duplication with Overlays: Uses a layering system — base config stays the same, and only differences are applied as overlays (defined in patch files)
 - No Templating Language Required: Works directly on YAML files — no new logic to learn
 - Built Into kubectl: Comes bundled with kubectl — no extra installation required

Code

```
# my-kustomize-app/  
# base/  
#   deployment.yaml  
#   service.yaml  
#   kustomization.yaml  
# overlays/  
#   dev/  
#       kustomization.yaml  
#       replica-patch.yaml  
#       secret.env
```

YAML / Kubernetes

```

# -----
# FILE: base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:1.21
          ports:
            - containerPort: 80
          envFrom:
            - secretRef:
                name: app-secret
              # WHAT: Mounts the generated secret
              # WHY: Passes environment variables securely

# -----
# FILE: base/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 80

# -----
# FILE: base/kustomization.yaml
resources:
  - deployment.yaml
  - service.yaml

# -----
# FILE: overlays/dev/replica-patch.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  # WHY: Dev needs fewer replicas than prod

# -----
# FILE: overlays/dev/secret.env
# WHAT: Defines secret key-value pairs
# WHY: Used by secretGenerator to create a Secret
# WHERE: This file must be in the same folder as kustomization.yaml
USERNAME=devuser
PASSWORD=devpass123

# -----
# FILE: overlays/dev/kustomization.yaml
resources:
  - ../../base

patchesStrategicMerge:
  - replica-patch.yaml

secretGenerator:
  - name: app-secret

```

```

envs:
  - secret.env
type: Opaque
# WHAT: Generates a Secret named `app-secret`
# WHY: Holds credentials securely in Kubernetes
# WHEN: Applied automatically on each `kubectl apply -k`
# TYPE:
# - Opaque (default): Generic key-value secret
# - kubernetes.io/dockerconfigjson: for Docker auth
# - kubernetes.io/tls: for TLS certs

# VARIATION:
# You can inline literals instead of a file:
# secretGenerator:
#   - name: app-secret
#     literals:
#       - USERNAME=devuser
#       - PASSWORD=devpass123

```

Q10

Compare Kustomize vs Helm

Aspect	Kustomize
What It Solves	Customize Kubernetes YAMLs for different environments without duplication
How It Works	Overlays and patches on base YAML files
Learning Curve	Easier — works directly with YAML
How to Run?	Native to kubectl — no extra tool needed
Templating Features	No templating — strict YAML-only patches
When to Use	When you want simple customization across environments

Q11

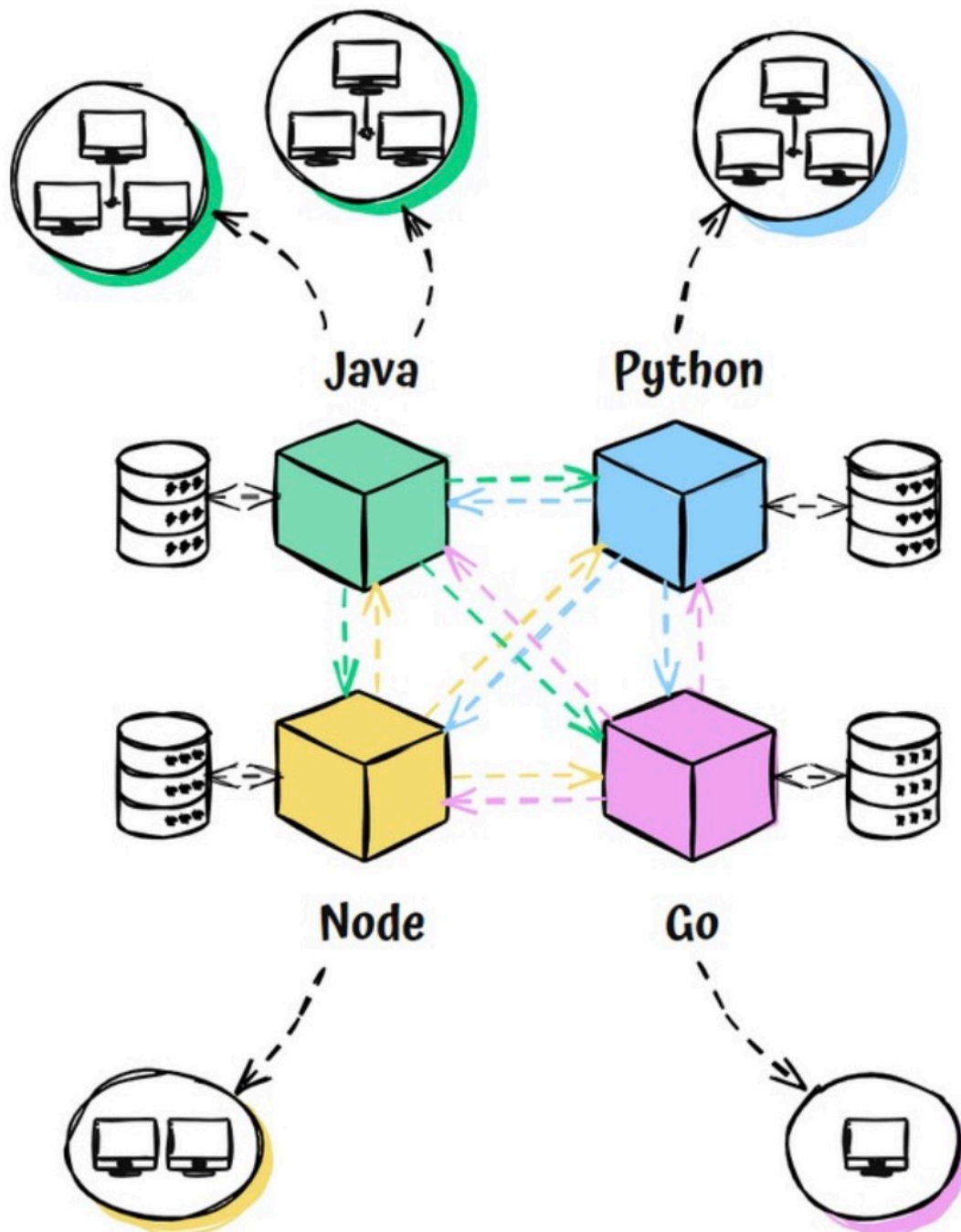
How can you implement GitOps with Argo CD in Kubernetes?

- What is Argo CD: A declarative GitOps continuous delivery tool for Kubernetes that automatically syncs cluster state with the desired state defined in Git repositories
 - Step 1 – Developer Commits to Git: A developer makes changes to Kubernetes manifests (or Helm/Kustomize configs) and commits them to a Git repository
 - Step 2 – Git Repository is Watched by Argo CD: Argo CD continuously watches the configured Git repository and the specified path for changes
 - Step 3 – Argo CD Detects Git Changes: When changes are detected (e.g., new deployment version), Argo CD compares the desired state from Git with the live state in the cluster
 - Step 4 – Argo CD Shows Out-of-Sync Status: If differences are found, Argo CD marks the application as “OutOfSync” and displays the diff via its CLI or Web UI
 - Step 5 – Sync Triggered Automatically or Manually: Based on the synchronization policy defined:
 - If set to auto-sync, Argo CD applies the new manifests immediately
 - If set to manual sync, a user must click "Sync" or use CLI to apply the change
 - Step 6 – Kubernetes Resources are Applied: Argo CD uses kubectl-like behavior to apply the updated manifests to the target cluster
 - Step 7 – Reconciliation and Health Checks: Argo CD continuously monitors the live state and performs health checks on resources (e.g., waiting for deployments to become available)
 - Step 8 – Application Marked as Healthy and Synced: Once all changes are successfully applied and resources are healthy, Argo CD marks the app as “Synced” and “Healthy”
 - Step 9 – Audit Trail is Maintained in Git: Since all changes go through Git, the full history of deployments, rollbacks, and configuration changes is stored in the Git log
 - Step 10 – Optional Rollback if Needed: If issues occur, the user can rollback to a previous Git commit and let Argo CD sync the cluster back to that known good state

CHAPTER 10

Kubernetes Interview - Diagrams

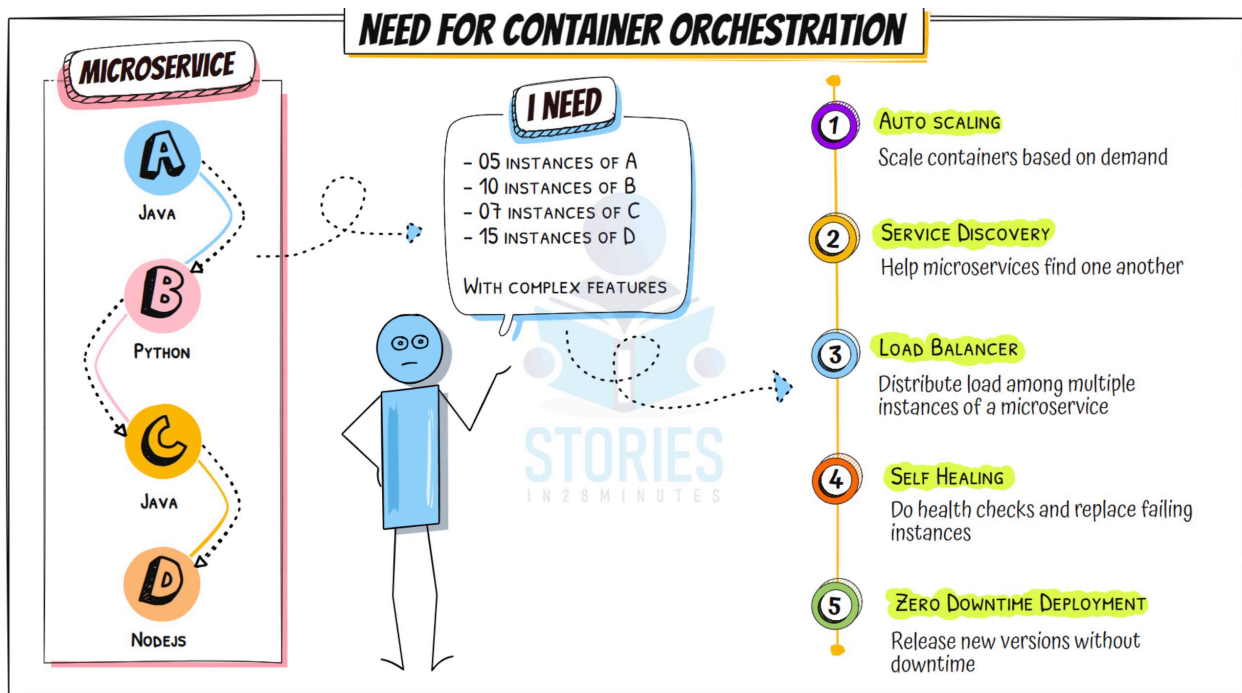
16 Questions & Answers



Microservices

Q2

Need For Container Orchestration



Need For Container Orchestration

Q3

Kubernetes Example Cluster

Code

```
graph TD
    subgraph Cluster
        MasterNode[Master Node]
        Worker1[Worker Node 1]
        Worker2[Worker Node 2]
        Worker3[Worker Node 3]
        Worker4[Worker Node 4]
    end

    MasterNode --> Worker1
    MasterNode --> Worker2
    MasterNode --> Worker3
    MasterNode --> Worker4
```

Q4

Kubernetes Node Components

Code

```

graph TB
%% Define common style for all boxes
classDef uniform fill:#ffffff,stroke:#333,stroke-width:1px,font-size:12px;

subgraph Cluster
  subgraph MasterNode["Master Node(s)"]
    A1("API Server"):::uniform
    A2("Scheduler"):::uniform
    A3("Controller Manager"):::uniform
    A4[("Etcd")]:::uniform
    A5("Cloud Controller Manager"):::uniform
    A6("Kubelet"):::uniform
    A7("Kube-Proxy"):::uniform
    A8("Container Runtime"):::uniform
  end

  subgraph WorkerNode1["Worker Node 1"]
    B11("Kubelet"):::uniform
    B12("Kube-Proxy"):::uniform
    B13("Container Runtime"):::uniform
  end

  subgraph WorkerNode2["Worker Node 2"]
    B21("Kubelet"):::uniform
    B22("Kube-Proxy"):::uniform
    B23("Container Runtime"):::uniform
  end

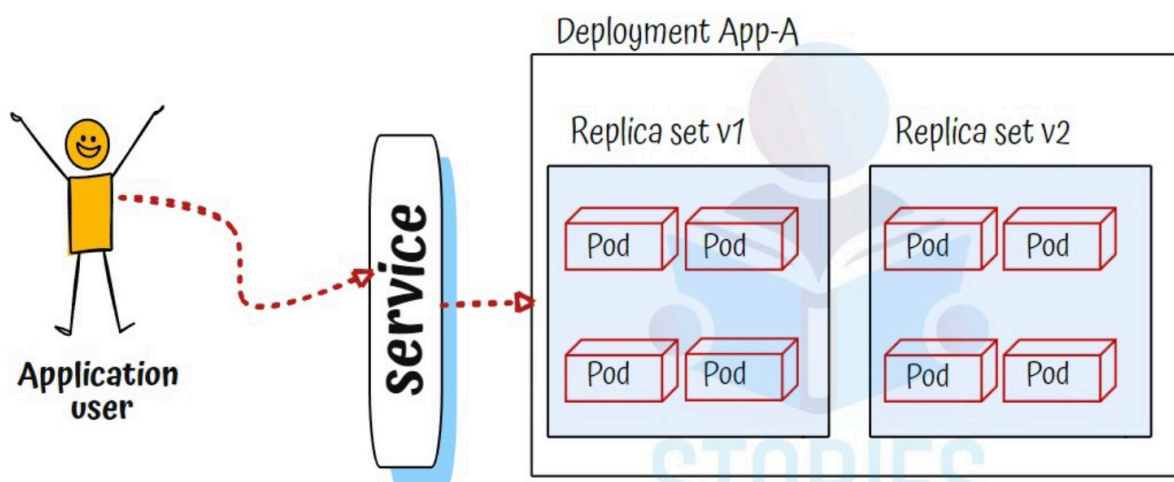
  subgraph WorkerNode3["Worker Node 3"]
    B31("Kubelet"):::uniform
    B32("Kube-Proxy"):::uniform
    B33("Container Runtime"):::uniform
  end
end

%% Colors for subgraphs
style MasterNode fill:#f0f8ff,stroke:#333,stroke-width:4px;
style WorkerNode1 fill:#f9f9f9,stroke:#333,stroke-width:2px;
style WorkerNode2 fill:#f9f9f9,stroke:#333,stroke-width:2px;
style WorkerNode3 fill:#f9f9f9,stroke:#333,stroke-width:2px;

```

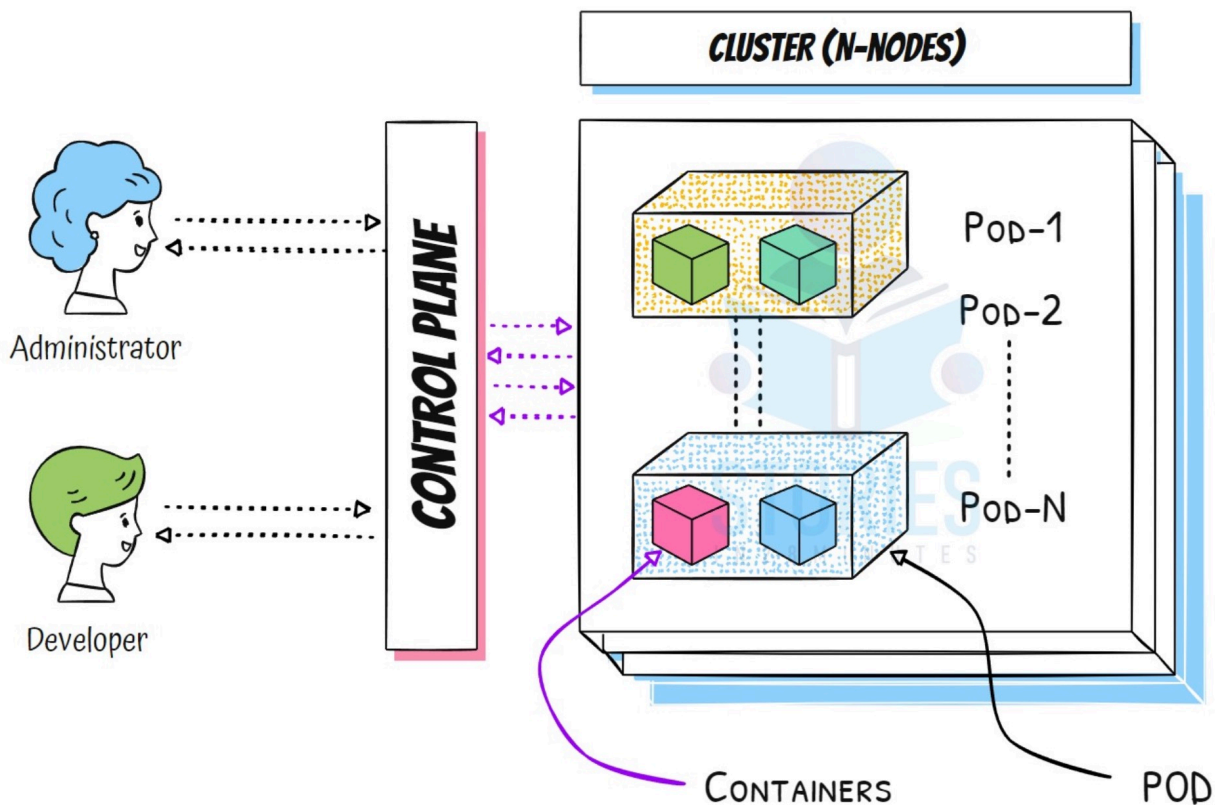
Q5

Kubernetes Logical View



Q6

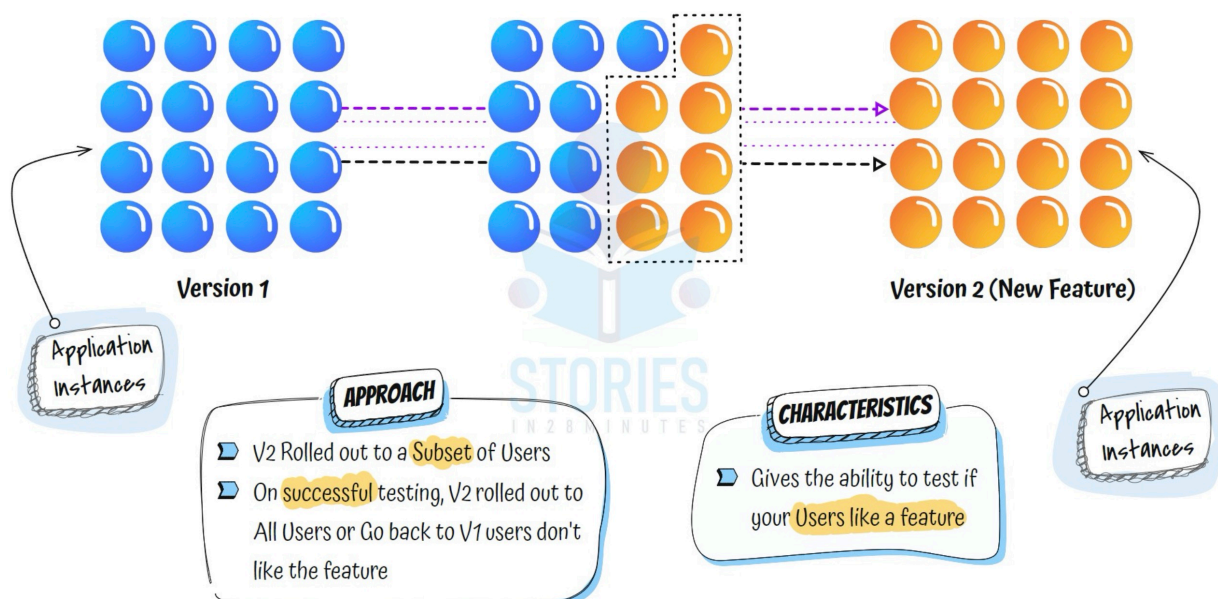
Kubernetes Physical View



Kubernetes Physical View

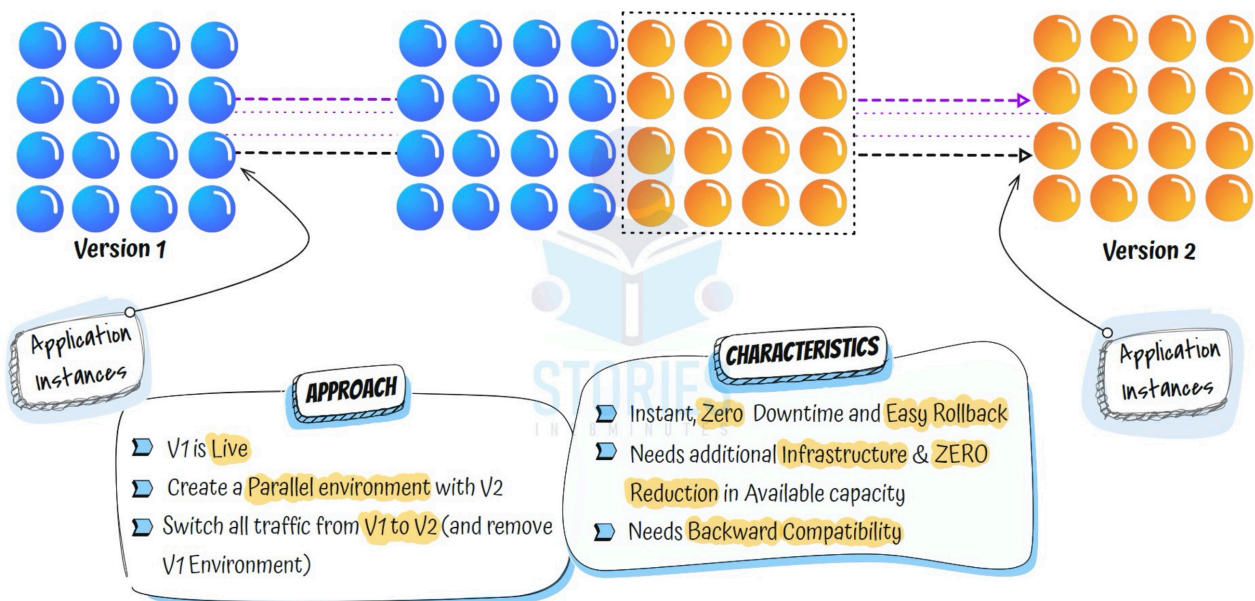
Q7

A/B Testing



Q8

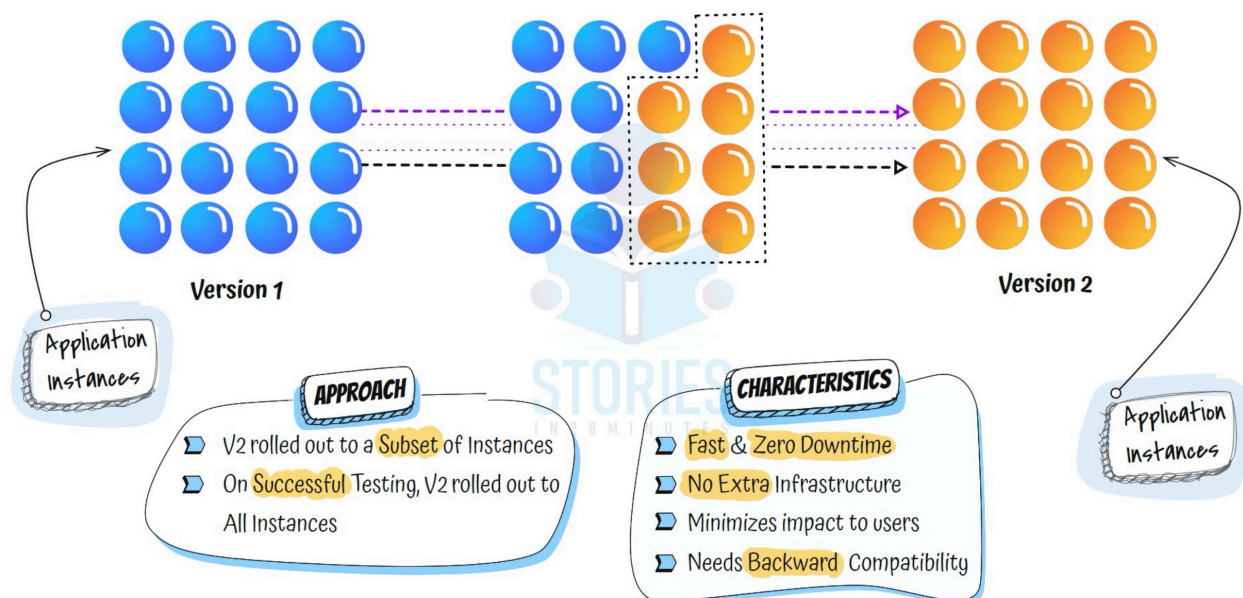
Blue Green Deployment



Blue Green Deployment

Q9

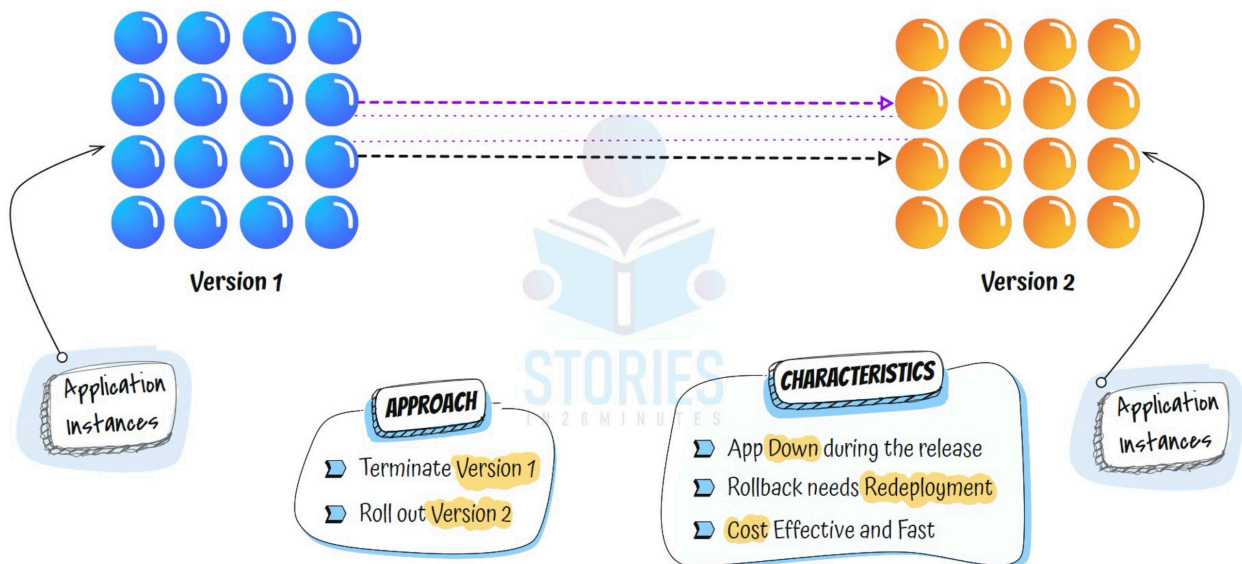
Canary Deployment



Canary Deployment

Q10

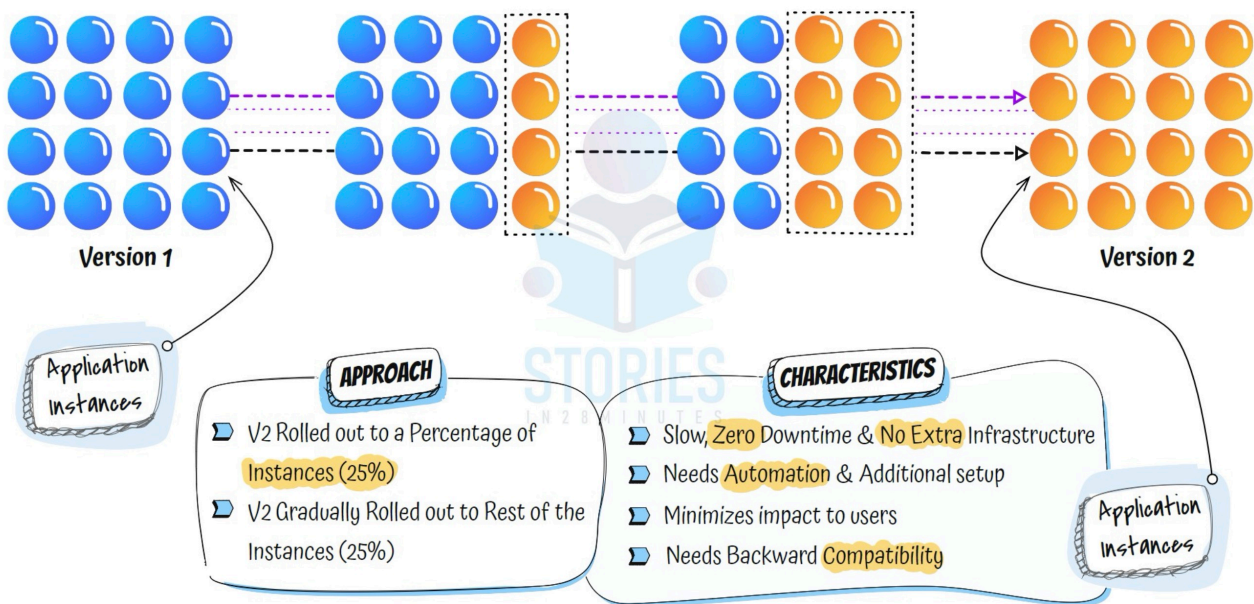
Recreate



Recreate

Q11

Rolling Updates

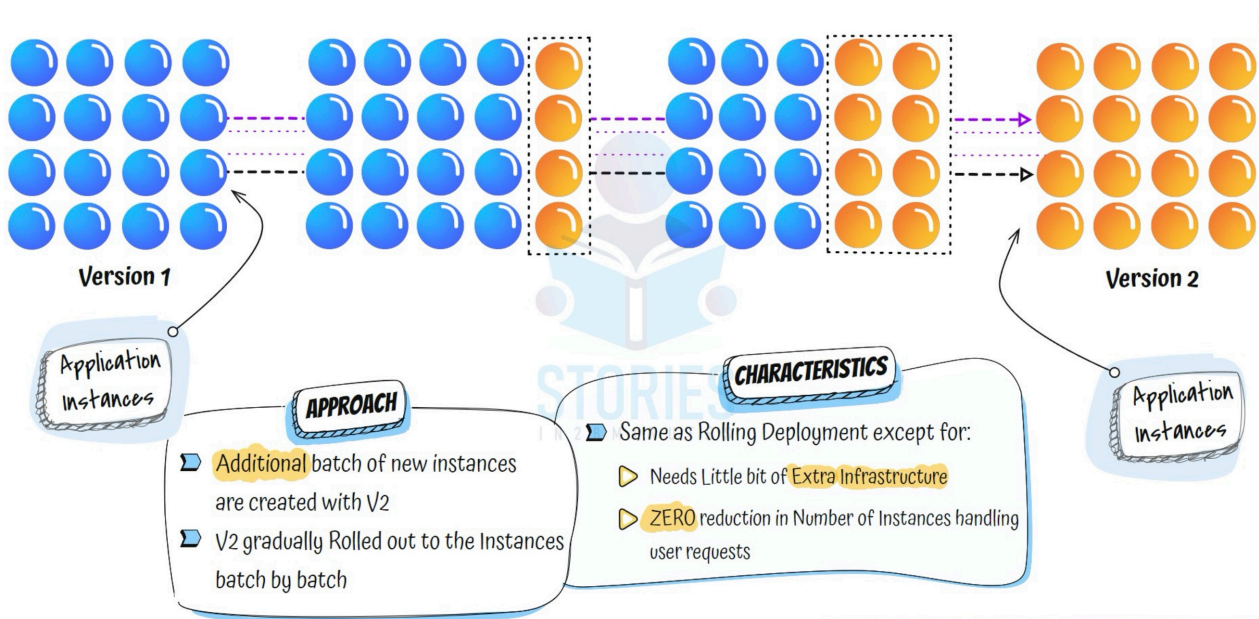


Rolling Updates

Q12

Rolling Updates with Additional Instances

Rolling Updates with Additional Instances



Q13

Service

Code

```
graph LR
  DEP[Deployment]
  RS[ReplicaSet]
  POD1[Pod1]
  POD2[Pod2]
  SVC[Service]

  SVC --> POD1
  SVC --> POD2
  DEP --> RS
  RS --> POD1
  RS --> POD2
```

Q14

Labels

kubernetes-labels

Labels

Q15

Storage

Code



```

graph LR
subgraph Storage
  subgraph StaticProvisioning ["Static Provisioning"]
    direction TB
    PVC1(PVC) -->|Matches| PV1(PV)
    PV1 -->|Mounted to| Pod1(Pod)
  end
  subgraph DynamicProvisioning ["Dynamic Provisioning"]
    direction TB
    PVC2(PVC) -->|Refers to| SC(StorageClass)
    SC -->|Provisions| PV2(PV)
    PV2 -->|Mounted to| Pod2(Pod)
  end
end

%% Colors for subgraphs
style Storage fill:#f0f8ff,stroke:#333,stroke-width:4px;
style StaticProvisioning fill:#f9f9f9,stroke:#333,stroke-width:2px;
style DynamicProvisioning fill:#f9f9f9,stroke:#333,stroke-width:2px;
  
```

Code

```
flowchart TB
    subgraph Nodes["Node"]
        PodA
        PodB
        PodC
        PodD
    end
    subgraph NodePool[" "]
        Nodes
    end
    subgraph Kubernetes_Cluster["Cluster"]
        NodePool
    end
    subgraph PodA["Pod"]
        A1["micro-a"]
        A2["side car"]
    end
    subgraph PodB["Pod"]
        B1["micro-b"]
        B2["side car"]
    end
    subgraph PodC["Pod"]
        C1["micro-c"]
        C2["side car"]
    end
    subgraph PodD["Kubelet"]
    end
```